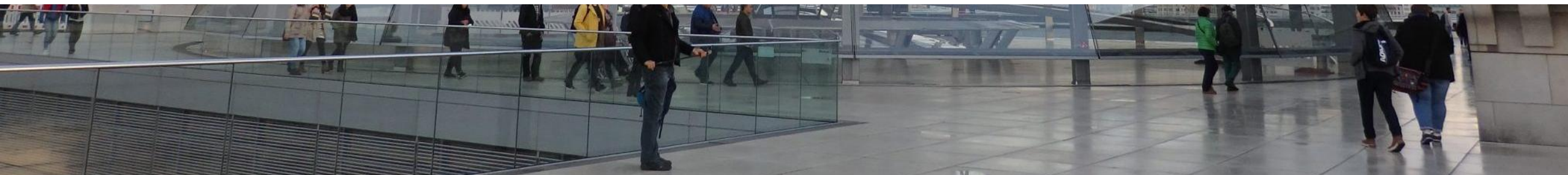




**PARAMETRIC DESIGN OF THE REICHSTAG DOME
VISUAL PROGRAMMING ADVANCED __LU YE**

**individualized
production**

RWTHAACHEN
UNIVERSITY



Content

- 1 Introduction
- 2 Concept of Parametric Design
- 3 Parametric Shape Generation
- 4 3D Model Development
- 5 2D Unfolding for Fabrication
- 6 5 Variants

Introduction

Reichstag Dome

- **Designed by:** Norman Foster
- **Location:** Berlin, Germany
- **Purpose:** Part of the **Reichstag building**, home to the **German Parliament** (Bundestag).

Why is it Relevant for This Project?

- The dome is a **great example of parametric architecture**.
- Its **geometry follows specific design rules**, making it ideal for **parametric modeling**.
- **Key elements** (façade, ramps, structural system) require **parametric flexibility**.



<https://rachelsruminations.com/reichstag-dome-berlin/>

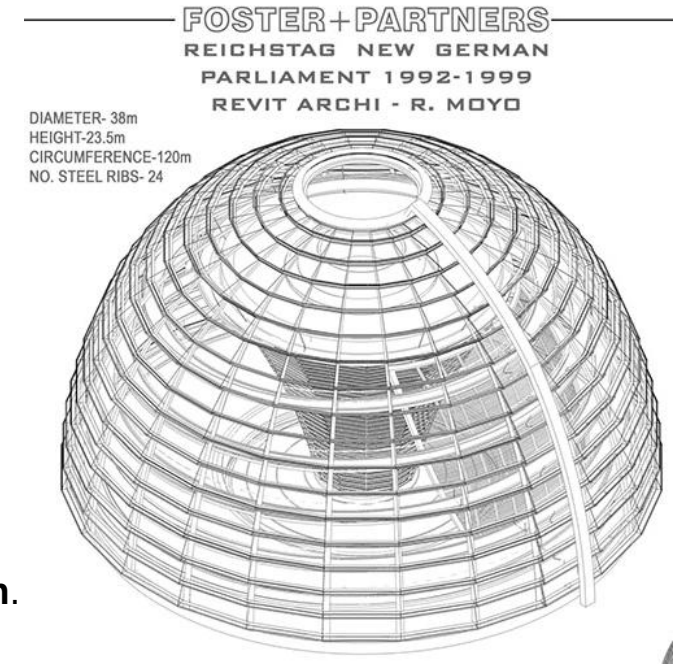
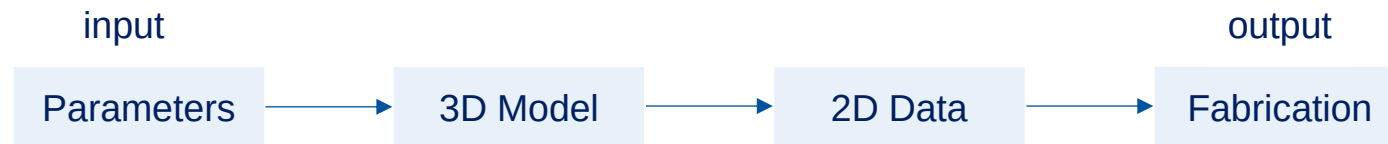
Project Objectives

Main Goal

- Develop a **parametric model** of the **Reichstag Dome** using **Grasshopper & Rhino**.
- Ensure the design is **rationalized and manufacturable** (not just free-form modeling).

Parametric Design Principles Applied in This Project

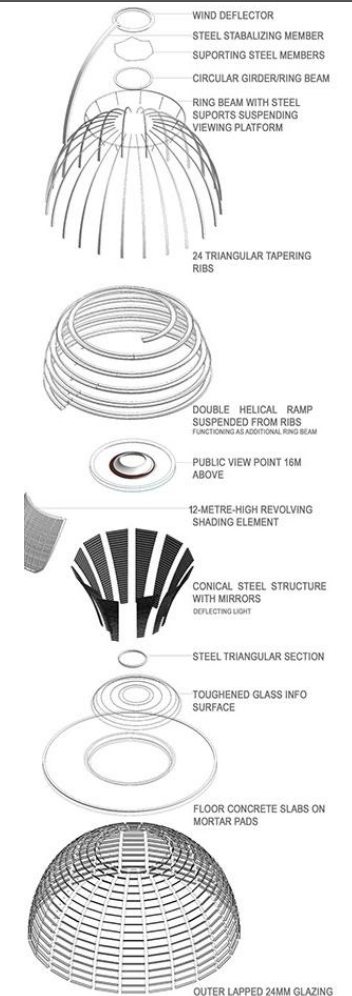
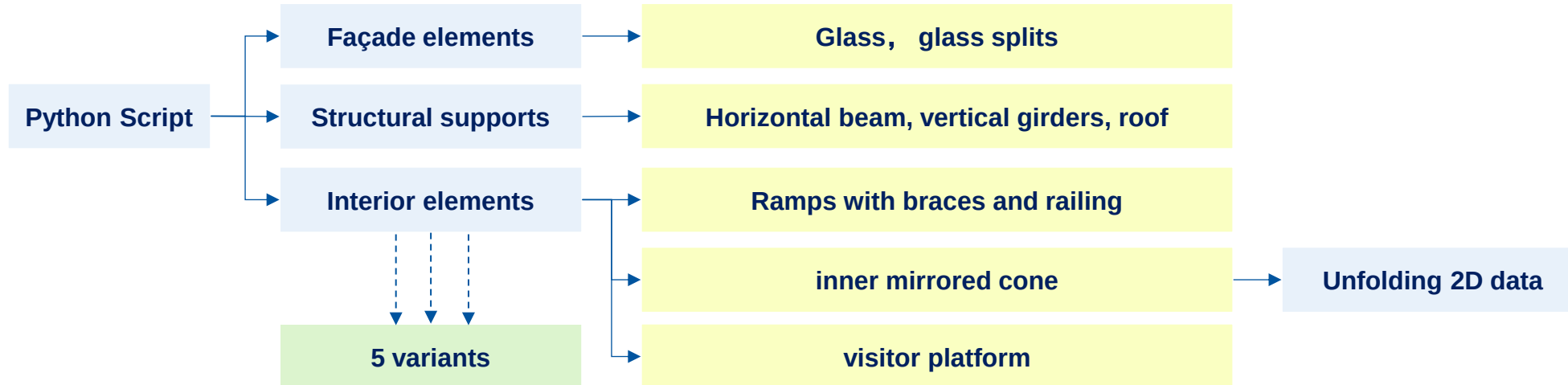
- **Rule-Based Geometry**: Dome shape follows **mathematical principles** rather than freehand design.
- **Scalability & Adaptability**: Design can be **adjusted dynamically** by modifying input parameters.
- **Digital Fabrication Readiness**: 2D data extraction makes the model **ready for real-world production**.



<https://www.behance.net/gallery/5420255/Reichstag-Dome-Structural-Exploration/>

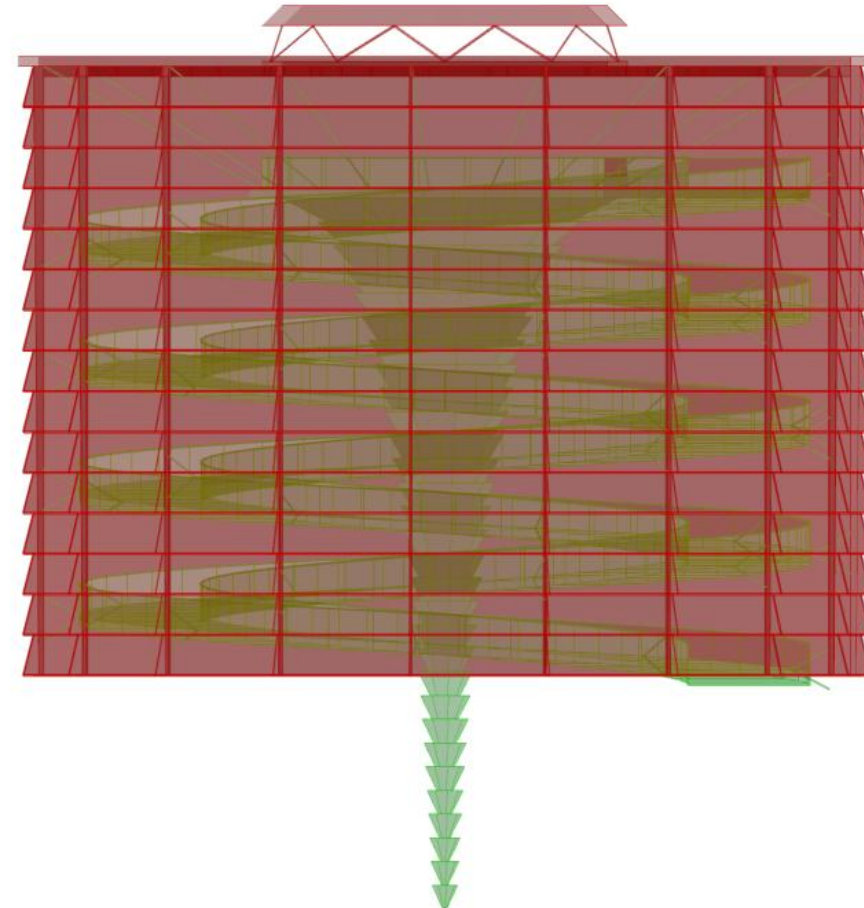
Key Objectives

1. Generate a parametric shape of the dome using a **Python Script in Grasshopper**.
2. Create a detailed 3D model with:
 - I. Façade elements
 - II. Structural supports (vertical girders & horizontal beams)
 - III. Ramps, inner mirrored cone, and visitor platform
3. Extract 2D fabrication data for selected construction elements (to prepare for laser cutting).
4. Create five design variants by modifying parameters



<https://www.behance.net/gallery/5420255/Reichstag-Dome-Structural-Exploration/>

1. Cyliner



Python Script

Parametric Shape Generation - Cylinder

```

1 import rhinoscriptsyntax as rs
2 import Rhino.Geometry as rg
3 import math
4 import ghpythonlib.treehelpers as gt
5
6 # Define the interval for height (vertical range) and circular angle (horizontal range) 定义范围
7 VD = rs.CreateInterval(0, 10) # Height range 高度范围
8 HD = rs.CreateInterval(0, 2 * math.pi) # Full circle range 圆周范围
9
10 # Function to generate evenly spaced values within a given range 生成范围的等分
11 def myRange(domain, steps):
12     start = domain[0]
13     end = domain[1]
14     step_size = (end - start) / steps
15     Values = [start + i * step_size for i in range(steps + 1)]
16     return Values
17
18 # Function to convert a list into a Data Tree structure 将普通列表转换为树结构 (Data Tree)
19 def Graft(List):
20     myTree = []
21     for i in range(len(List)):
22         myTree.append([List[i]]) # Each item is placed in its own branch 每个元素单独放入一个
23     return gt.list_to_tree(myTree)
24
25 # Generate vertical and horizontal ranges 生成垂直和水平分割范围
26 V_Steps = int(V_Steps) # Vertical divisions
27 H_Steps = int(H_Steps) # Horizontal divisions
28
29 VD_Range = myRange(VD, V_Steps)
30 HD_Range = myRange(HD, H_Steps)
31
32 # Convert ranges to Data Tree structures 转换为树结构
33 VD_Tree = Graft(VD_Range)
34 HD_Tree = Graft(HD_Range)

```

```

35 # Function to calculate X coordinates for each point on the circular layers 计算 X 坐标
36 def X_Coordinates(radius, u, v):
37     myTreeList = []
38     for i in range(u.BranchCount):
39         myBranchList = u.Branch(i)
40         myTempList = []
41         for k in range(len(v)):
42             x = radius * math.cos(v[k])
43             myTempList.append(x)
44         myTreeList.append(myTempList)
45     return gt.list_to_tree(myTreeList)
46
47 # Function to calculate Y coordinates for each point on the circular layers 计算 Y 坐标
48 def Y_Coordinates(radius, u, v):
49     myTreeList = []
50     for i in range(u.BranchCount):
51         myBranchList = u.Branch(i)
52         myTempList = []
53         for k in range(len(v)):
54             y = radius * math.sin(v[k])
55             myTempList.append(y)
56         myTreeList.append(myTempList)
57     return gt.list_to_tree(myTreeList)
58
59 # Function to calculate Z coordinates for each point along the height 计算 Z 坐标
60 def Z_Coordinates(height, u, v):
61     myTreeList = []
62     for i in range(u.BranchCount):
63         myBranchList = u.Branch(i)
64         myTempList = []
65         for k in range(len(v)):
66             z = myBranchList[0] * height / u.Branch(u.BranchCount - 1)[0] # Normalize Z coord
67             myTempList.append(z)
68         myTreeList.append(myTempList)
69     return gt.list_to_tree(myTreeList)
70

```

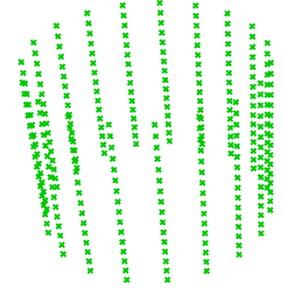
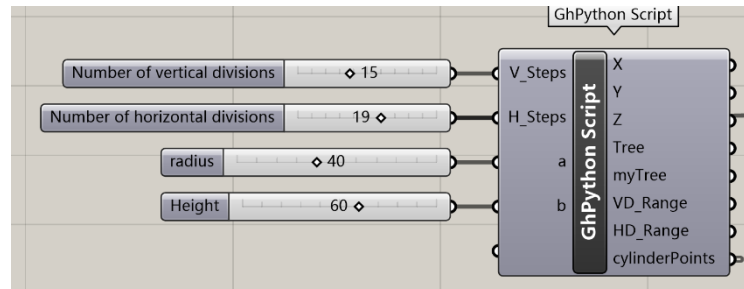
```

74 # Input values for the radius and height of the shape 用函数生成点云
75 radius = float(a)
76 height = float(b)
77
78 # Generate X, Y, Z coordinate trees
79 X = X_Coordinates(radius, VD_Tree, HD_Range)
80 Y = Y_Coordinates(radius, VD_Tree, HD_Range)
81 Z = Z_Coordinates(height, VD_Tree, HD_Range)
82
83 # Function to generate 3D points from the coordinate trees 生成点
84 def genPoints(x_tree, y_tree, z_tree):
85     myPointTree = []
86     count_1 = x_tree.BranchCount # Number of branches
87     count_2 = len(x_tree.Branch(0)) # Number of points in each branch
88
89     for i in range(count_1):
90         myTempList_X = x_tree.Branch(i)
91         myTempList_Y = y_tree.Branch(i)
92         myTempList_Z = z_tree.Branch(i)
93         myPointList = []
94
95         for k in range(count_2):
96             myTempPoint = rg.Point3d(myTempList_X[k], myTempList_Y[k], myTempList_Z[k])
97             myPointList.append(myTempPoint)
98
99         myPointTree.append(myPointList)
100
101     return gt.list_to_tree(myPointTree)
102
103 # Generate the final tree of points for the cylindrical shape 输出点云
104 cylinderPoints = genPoints(X, Y, Z)
105

```

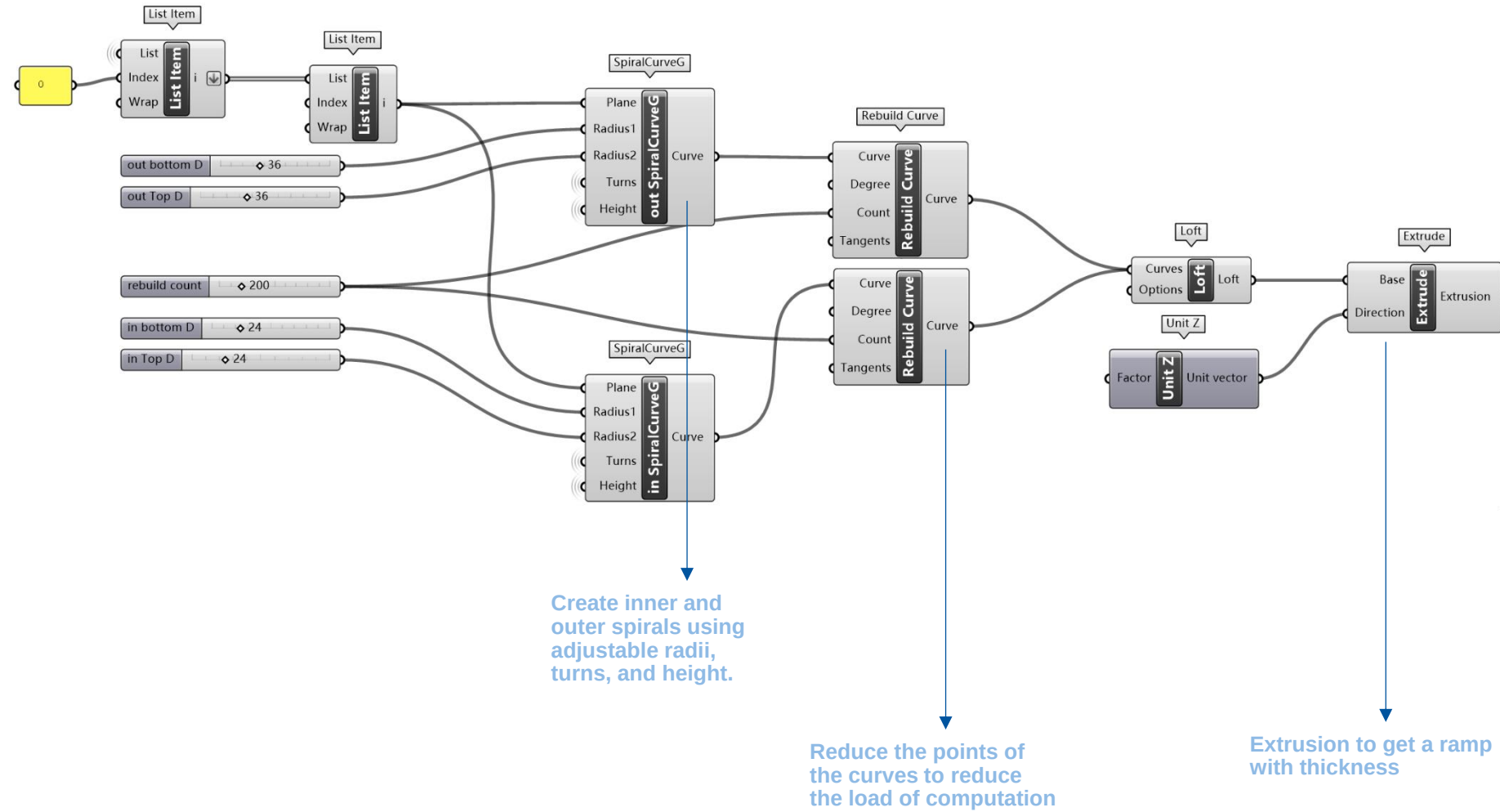
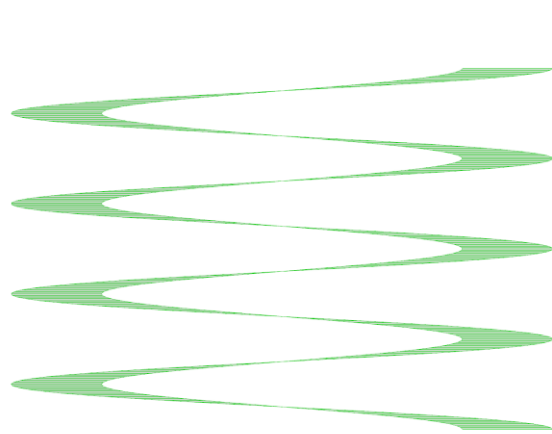


1. Generate a parametric point cloud by discretizing height and circular divisions.
2. Adjust the base and top radius dynamically to calculate X, Y, and Z coordinates using mathematical functions.
3. Output a structured data tree of points for creating a mesh or NURBS surface in Grasshopper.

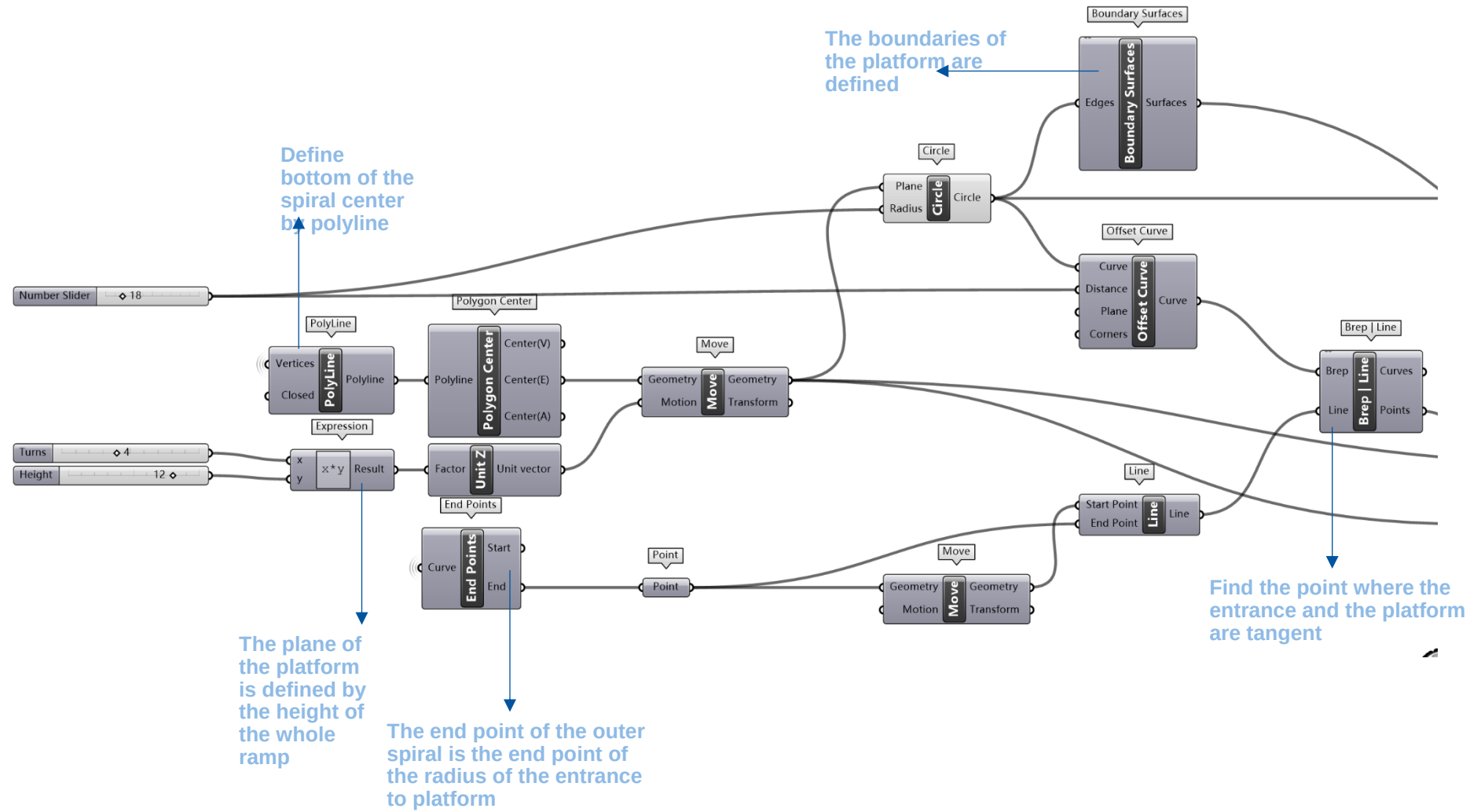
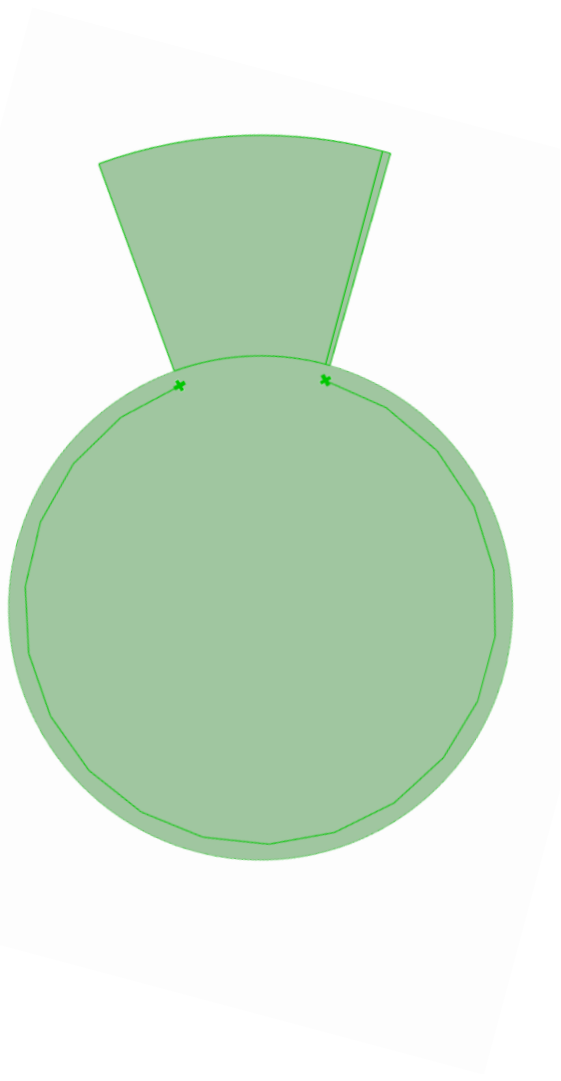


Interior elements

Ramp

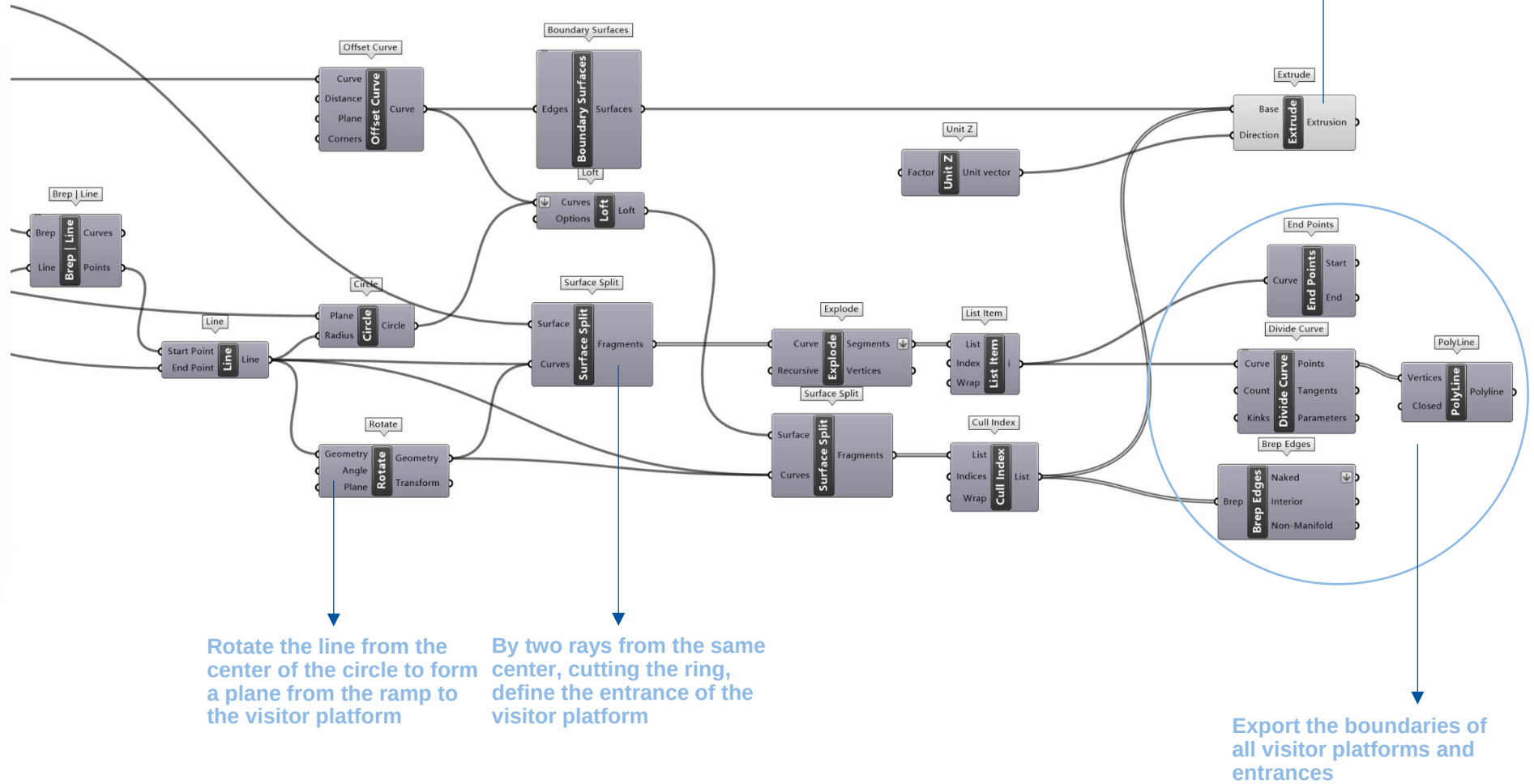
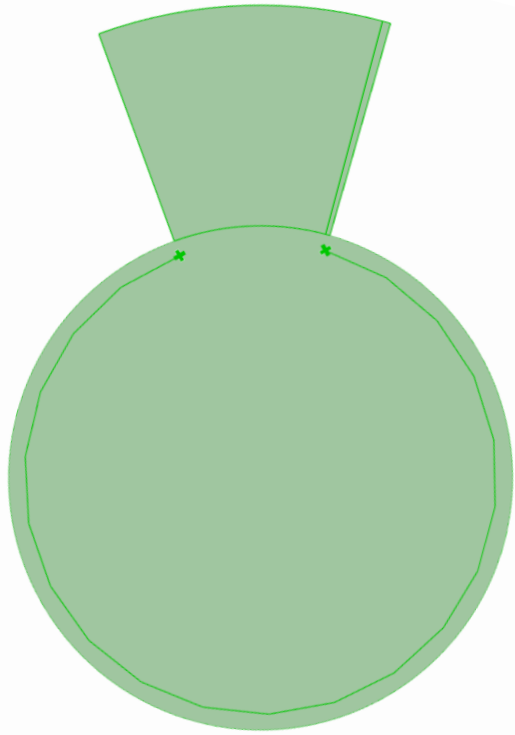


Visitor platform_1



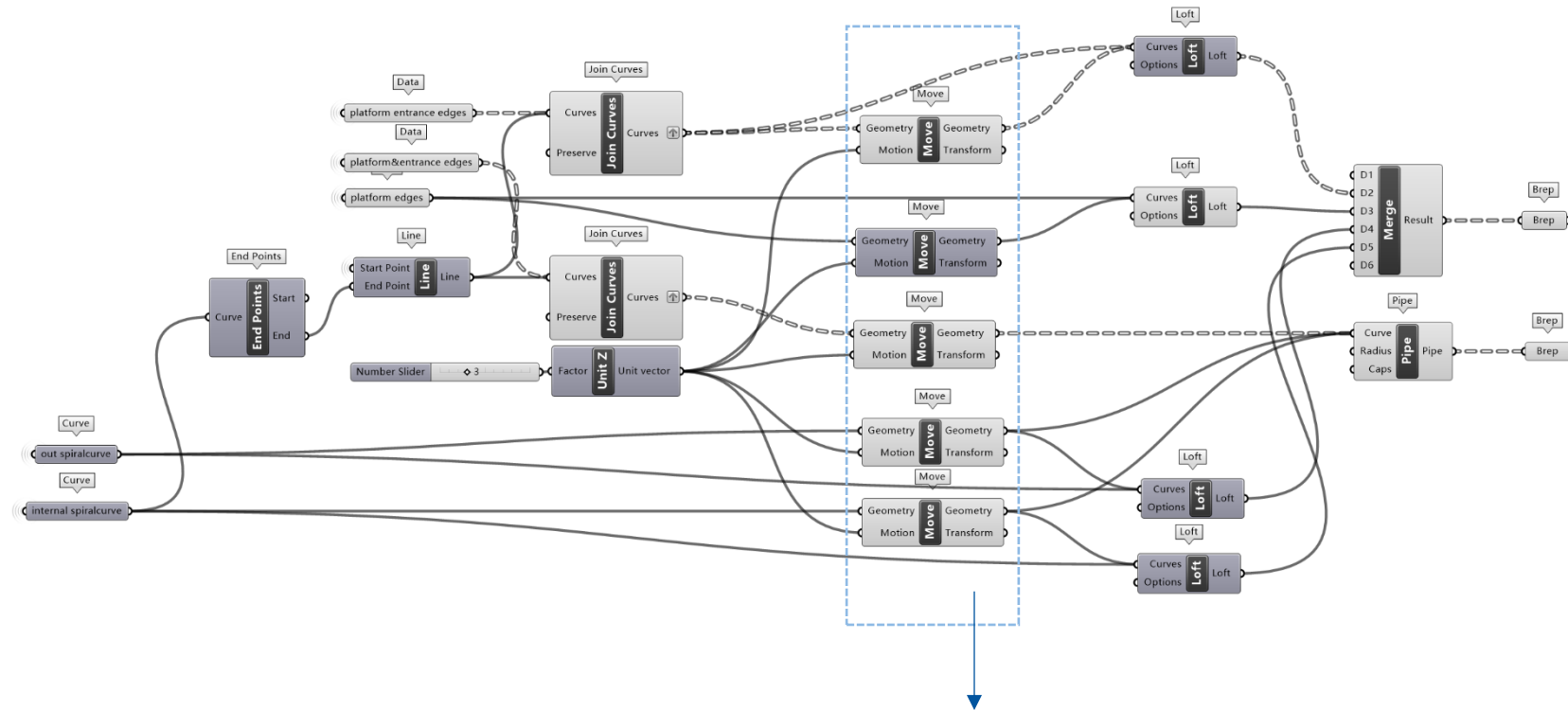
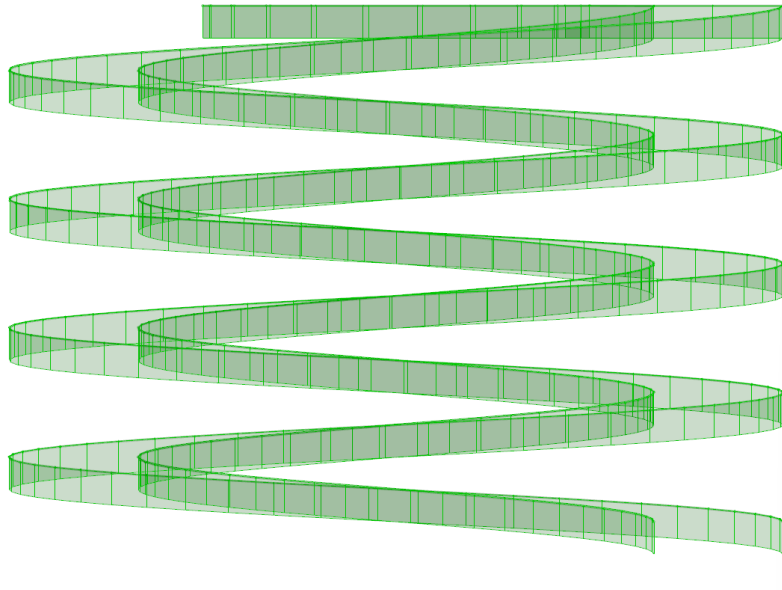
Interior elements

Visitor platform_2



Interior elements

Ramps and Visitor Platform's railing

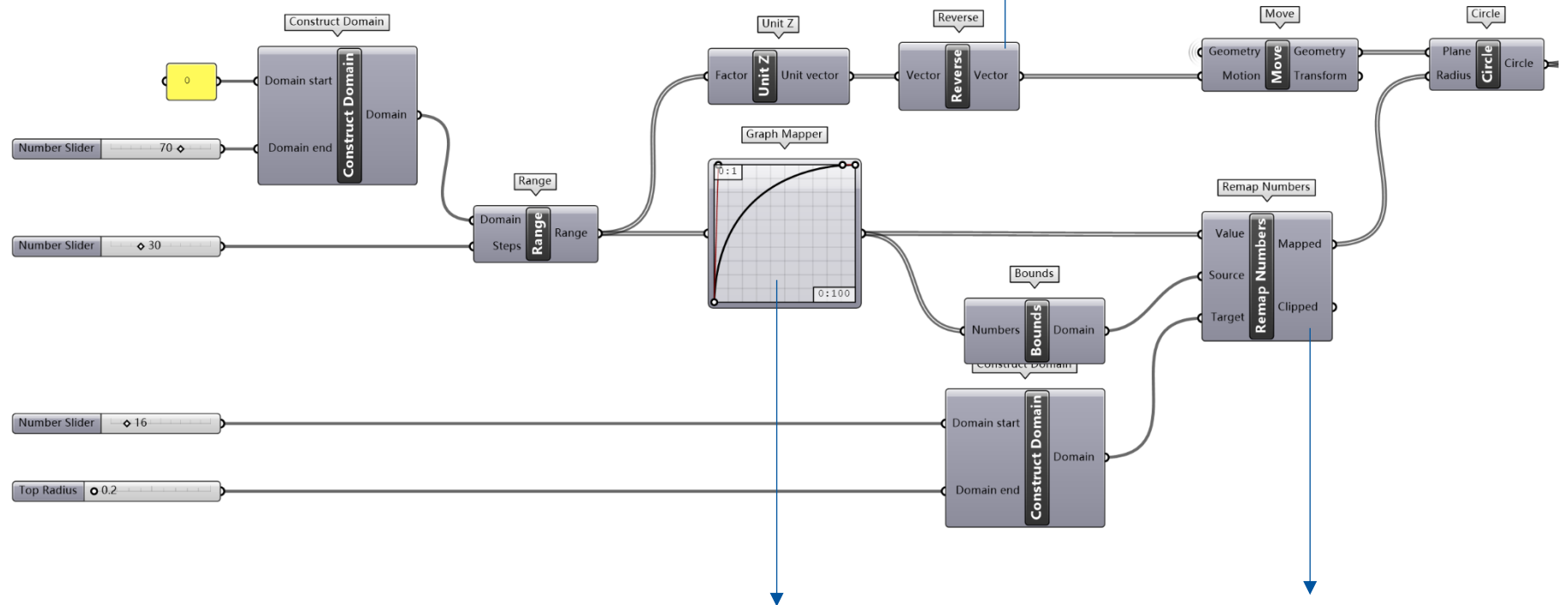
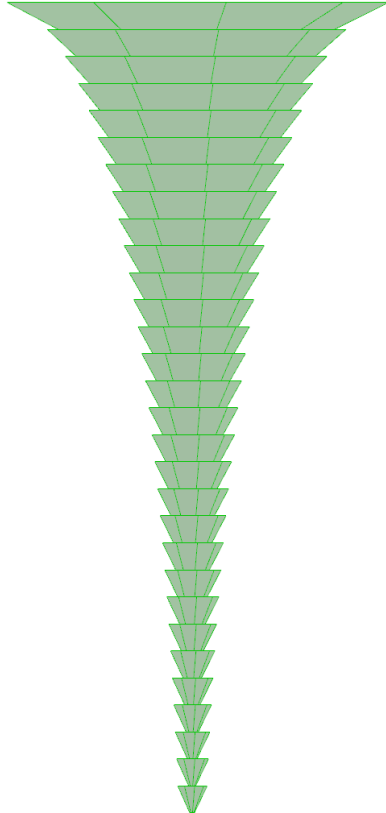


Move upwards and
create a synthetic
curve to construct the
handrail and glass
railing

1123121312

Interior elements

Inner mirrored cone_1



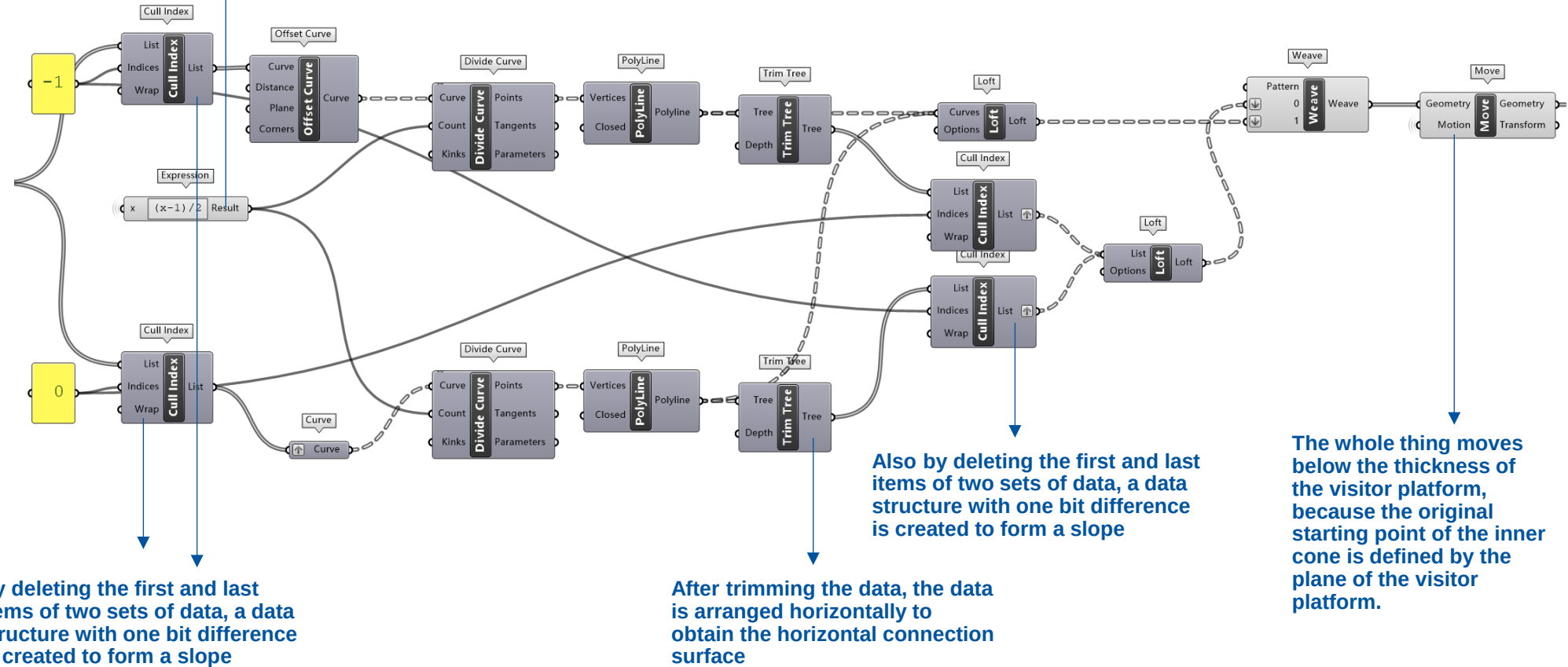
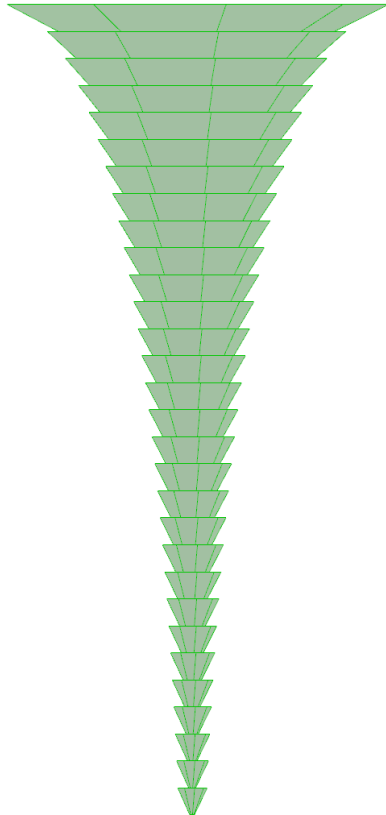
After the reversal,
the cone is
downward

Construct a rapidly
shrinking anti-cone
through a function

Scales a set of values
from one range to
another while
maintaining their
relative proportions.

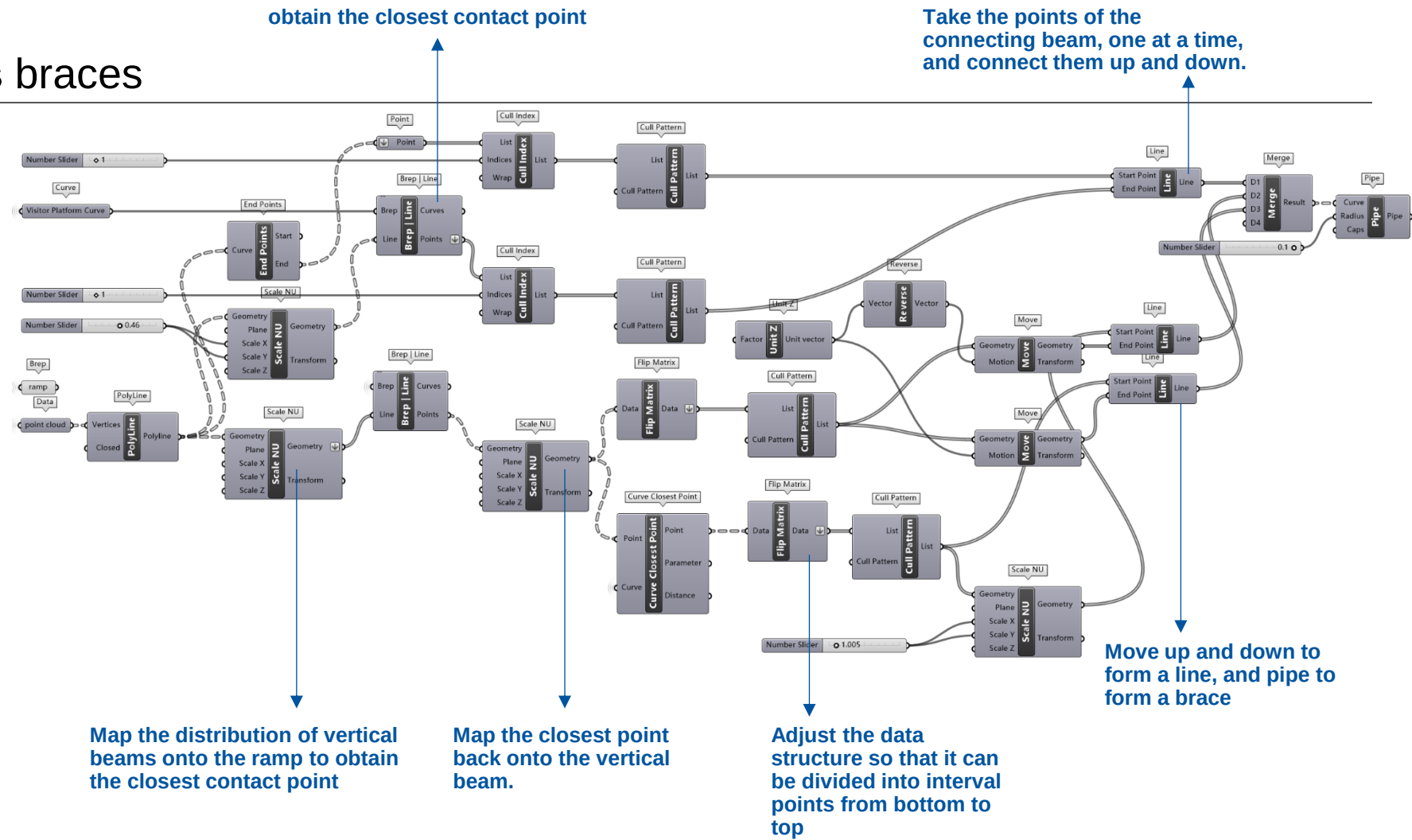
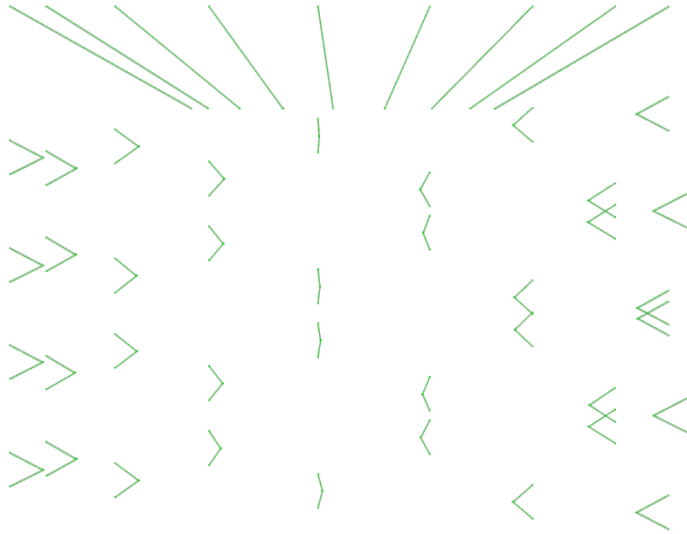
Inner mirrored cone_2

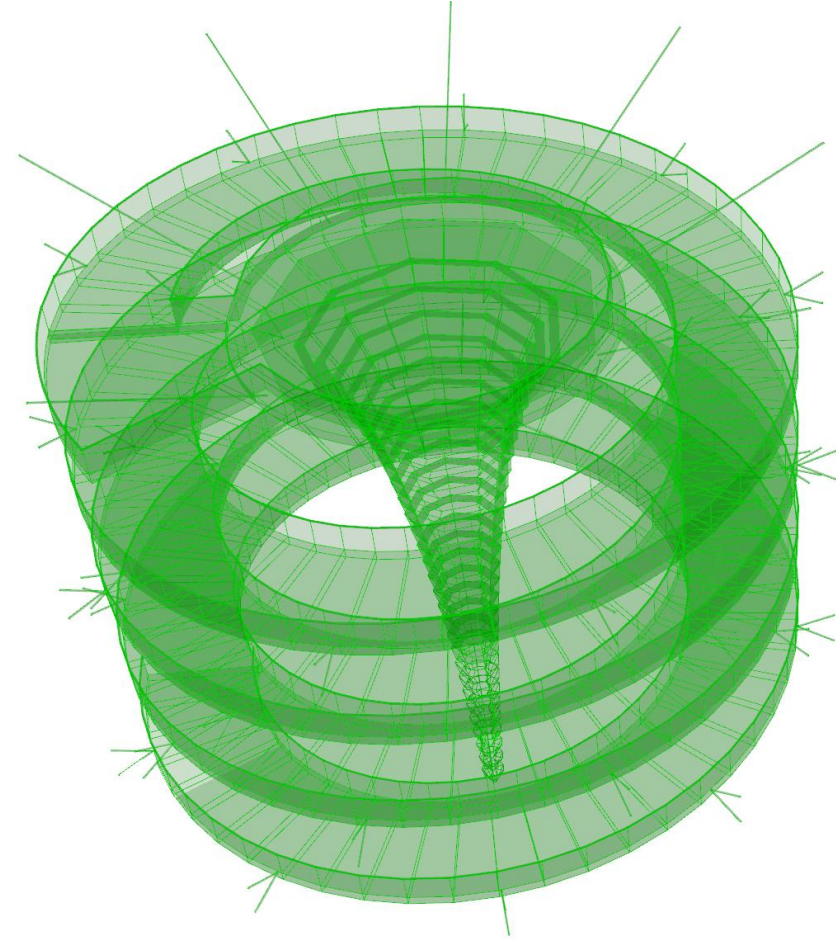
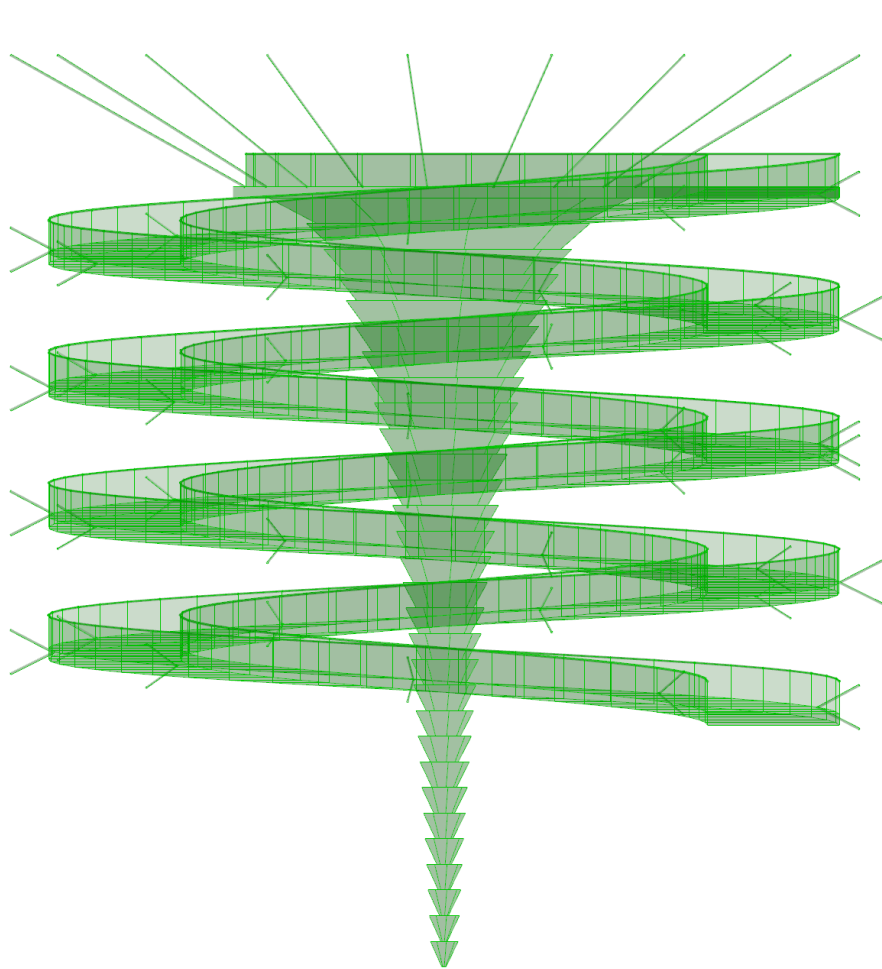
The number of faces that the inner cone is divided into is defined by Number of vertical divisions



Interior elements

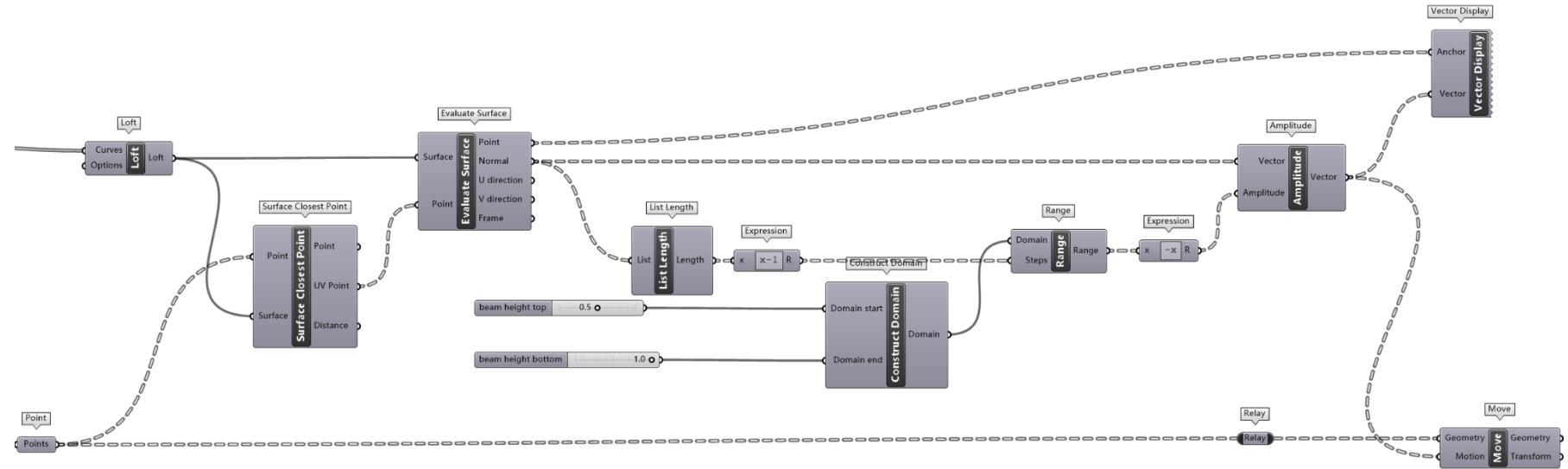
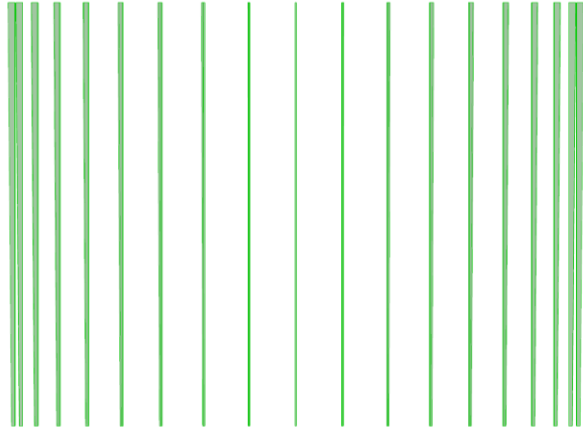
Ramps and Visitor Platform's braces





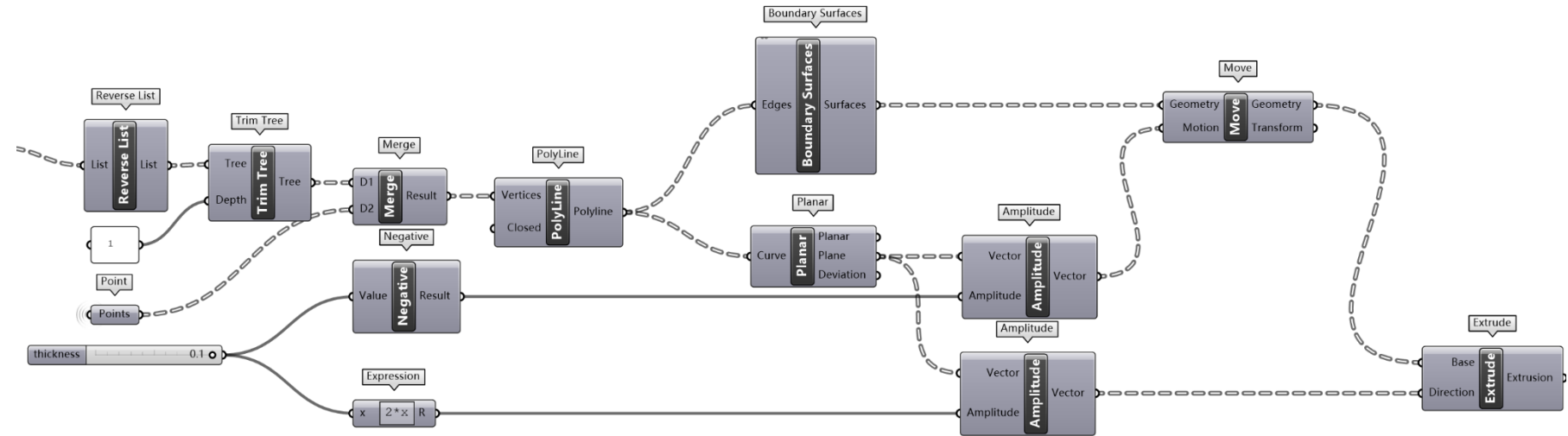
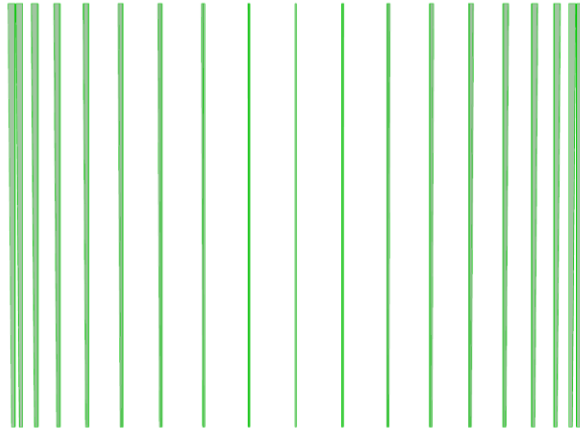
Structural supports

Vertical girders_1



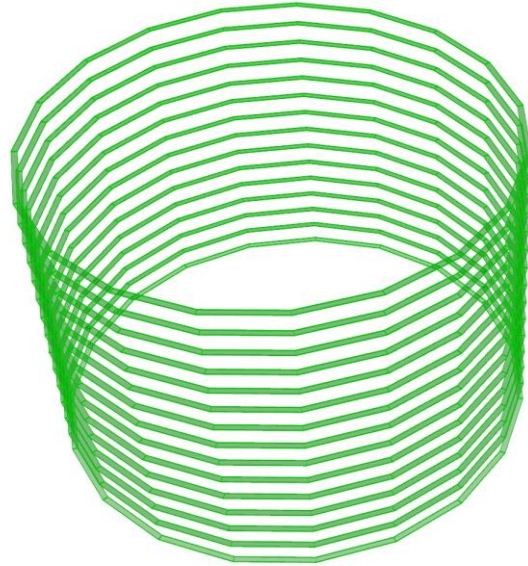
Structural supports

Vertical girders_2

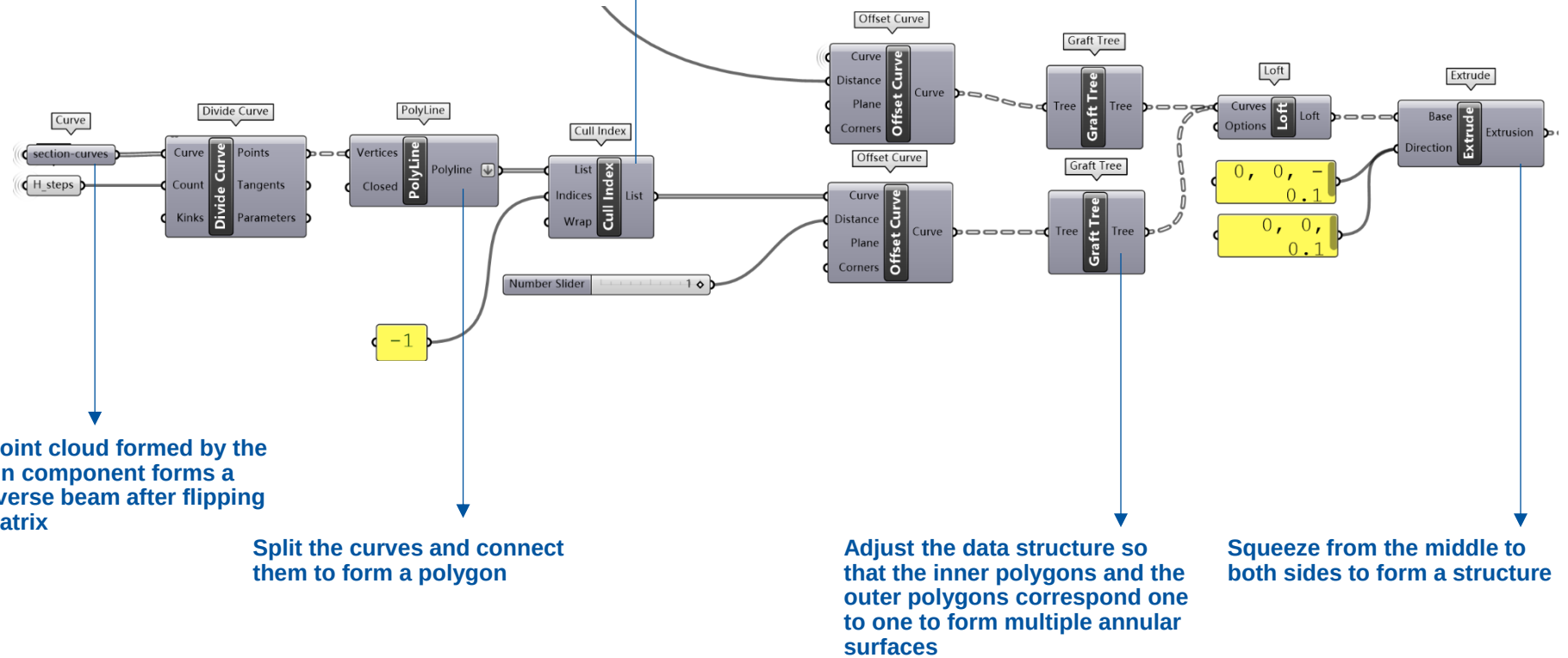


Structural supports

horizontal beams

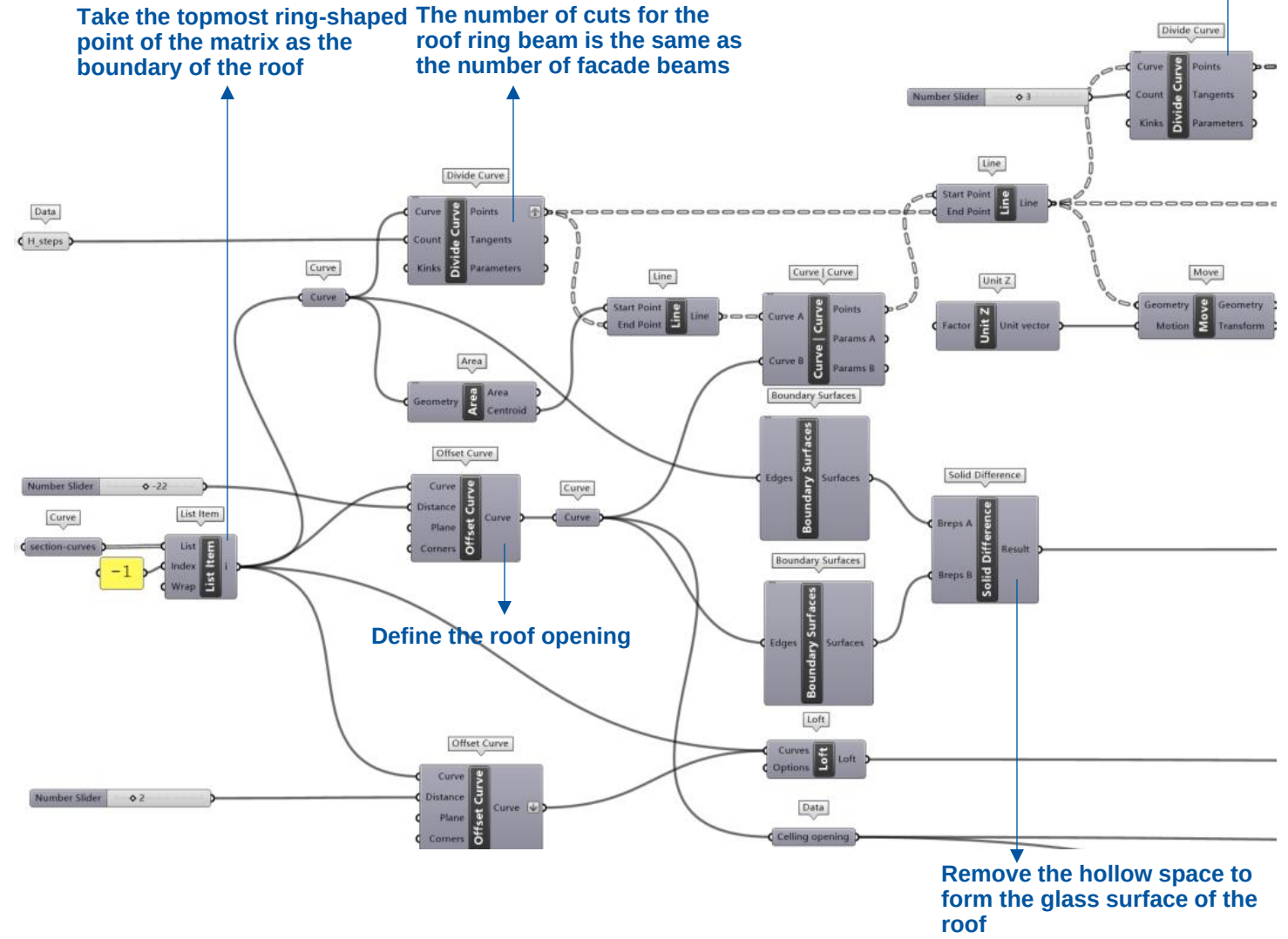
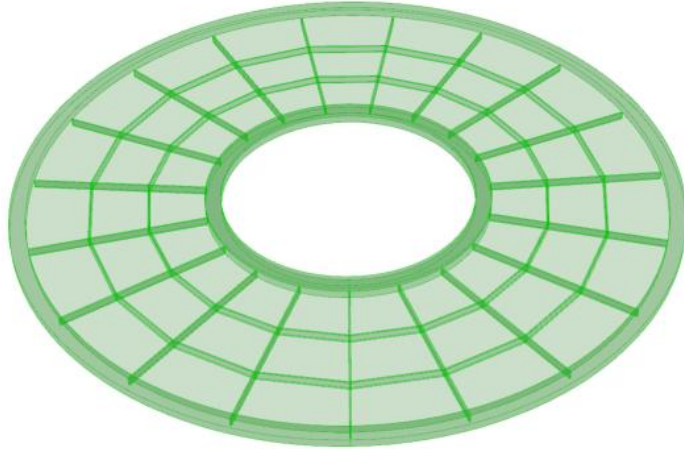


The point cloud formed by the python component forms a transverse beam after flipping the matrix

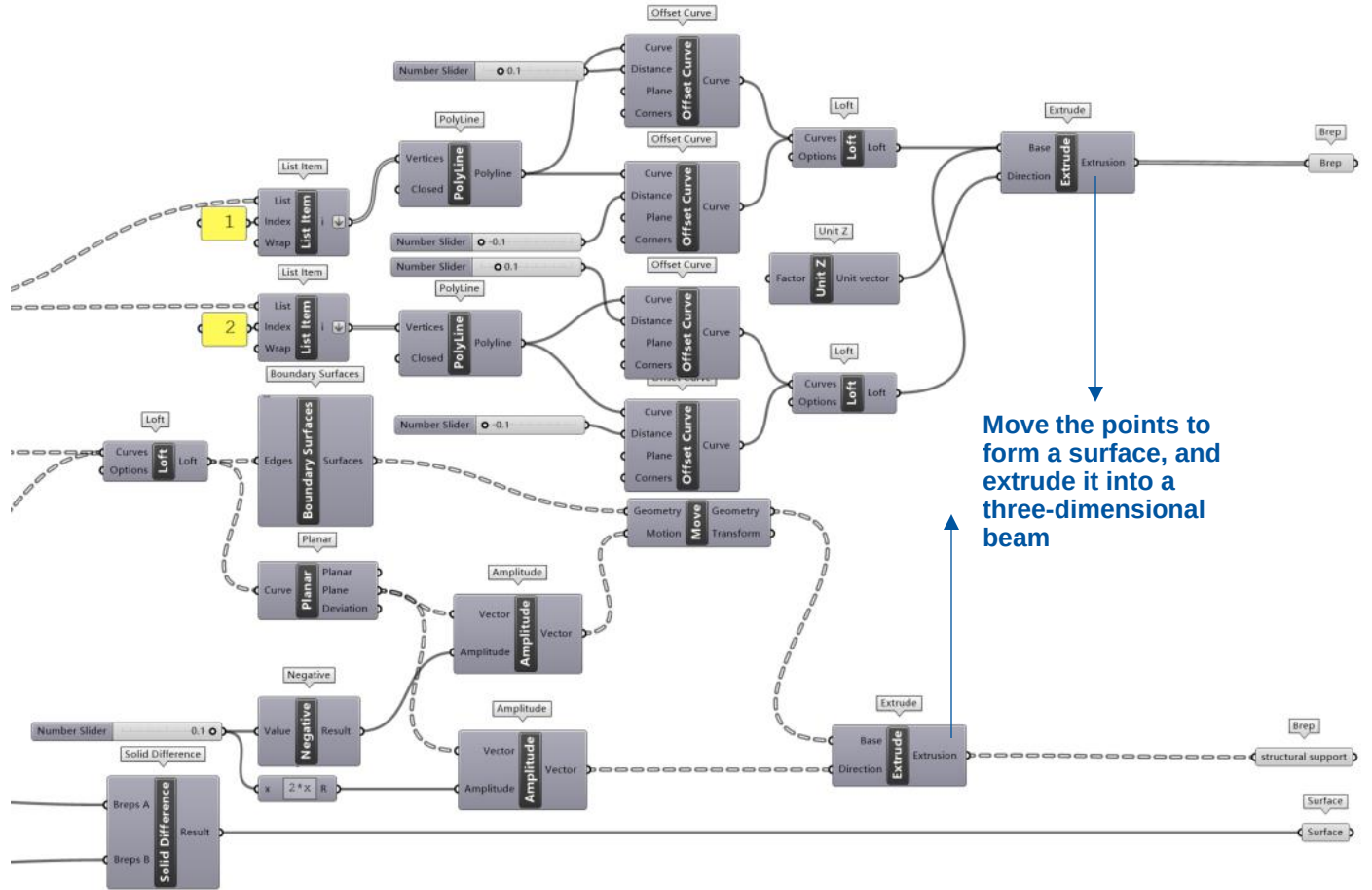


Structural supports

Roof structure_1

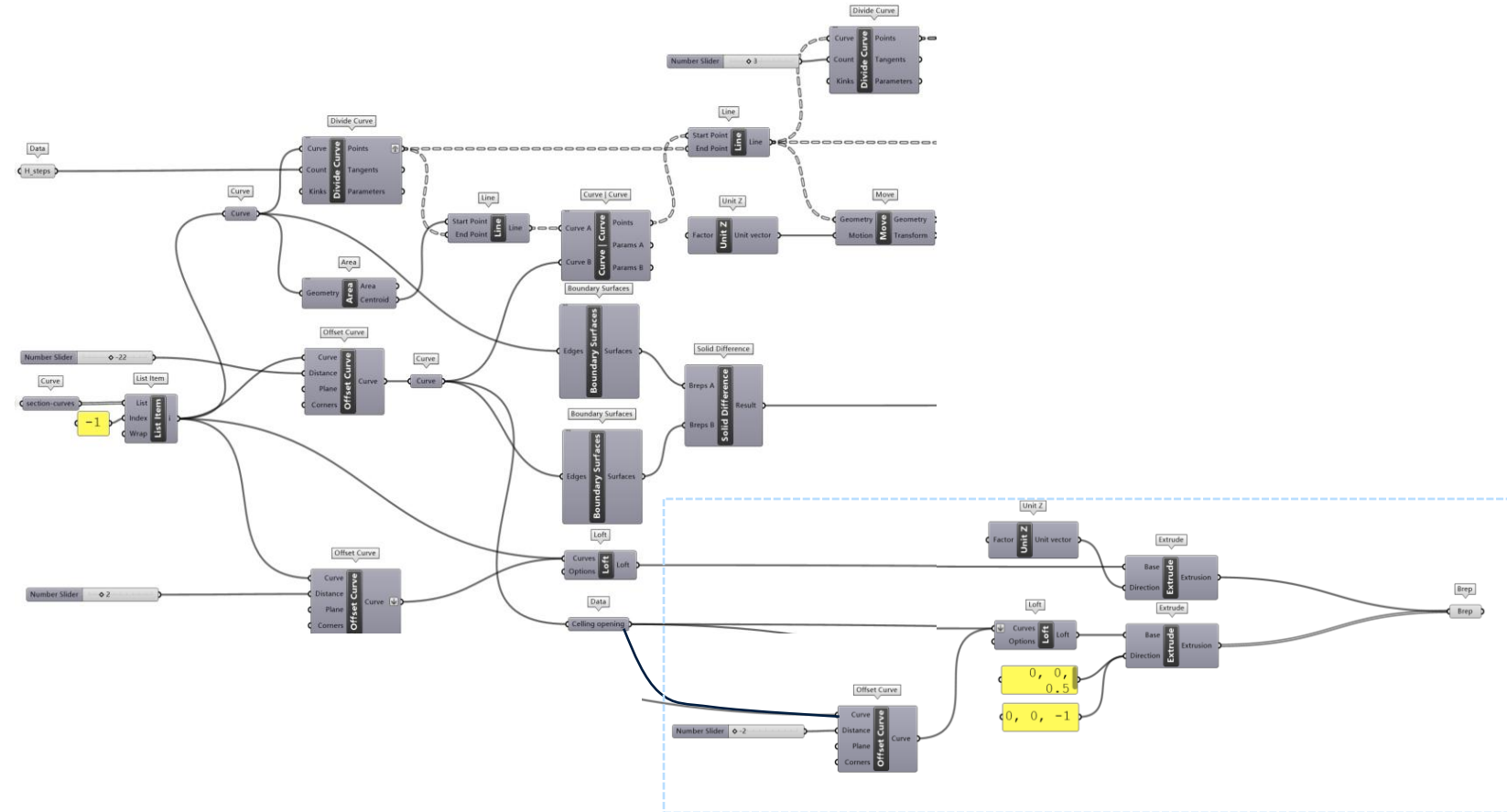
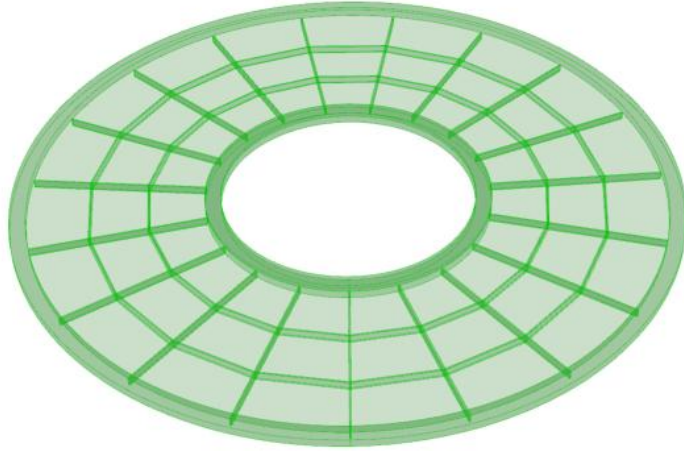


Roof structure_2



Structural supports

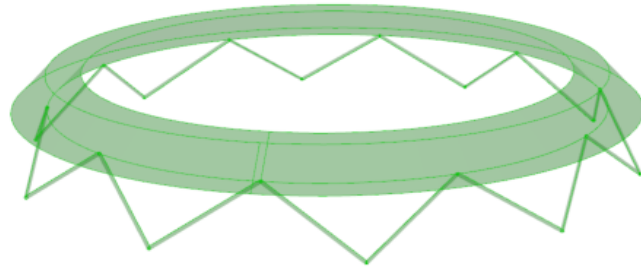
Roof structure_3



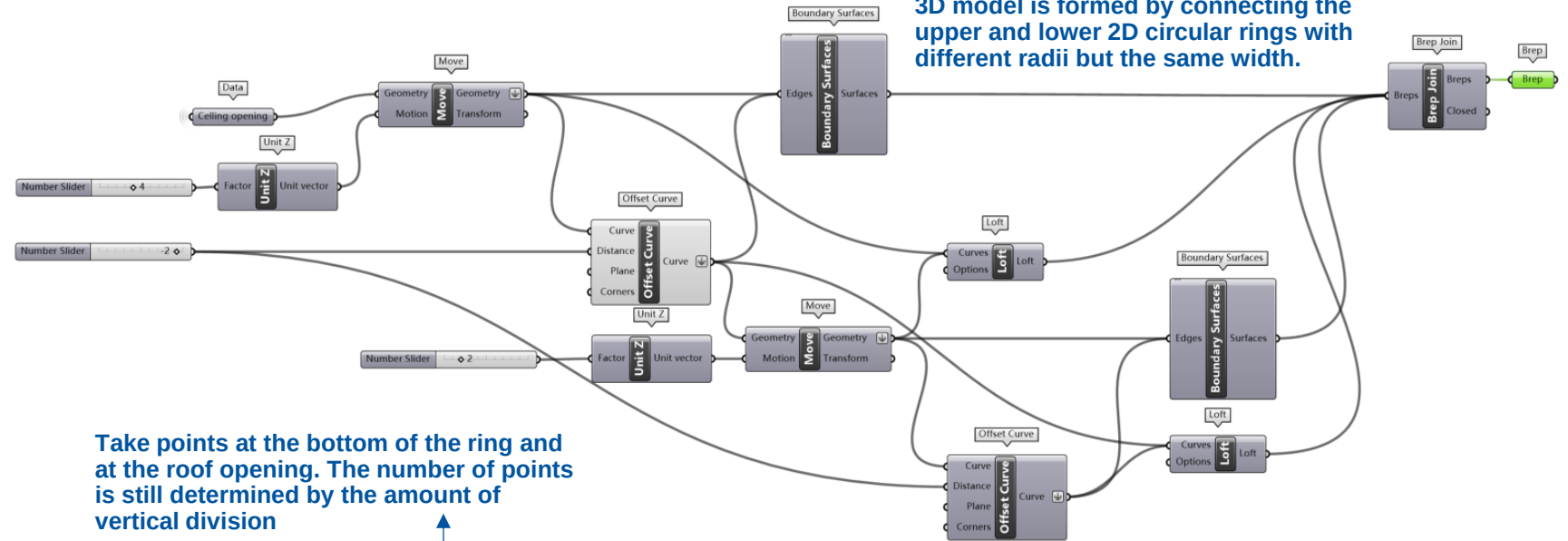
Lead out the ring beam
separately and make it thicker

Structural supports

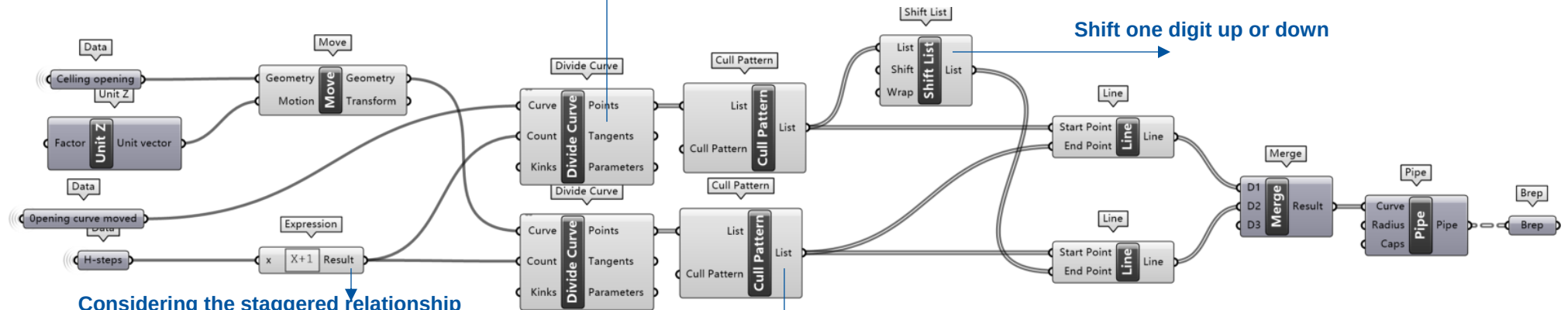
Ring Top



This is a circular ring tilted inwards. The 3D model is formed by connecting the upper and lower 2D circular rings with different radii but the same width.



Take points at the bottom of the ring and at the roof opening. The number of points is still determined by the amount of vertical division

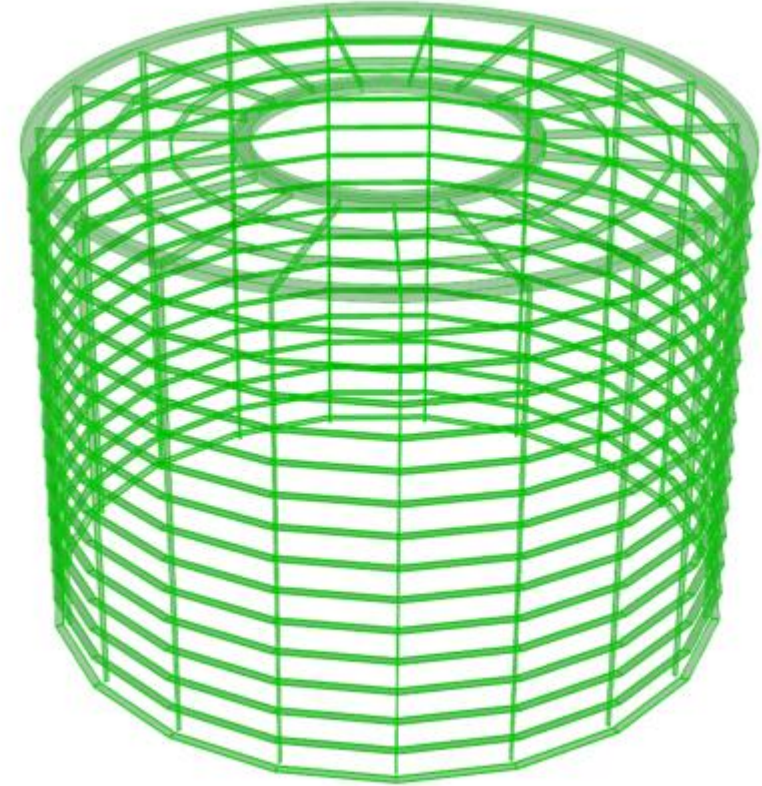
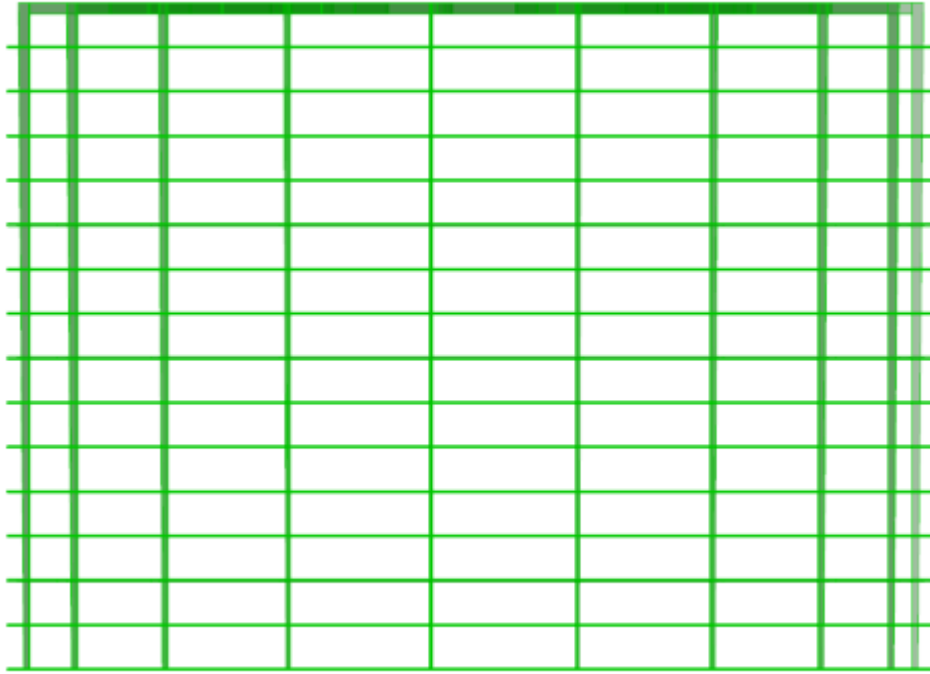


Shift one digit up or down

1123121312

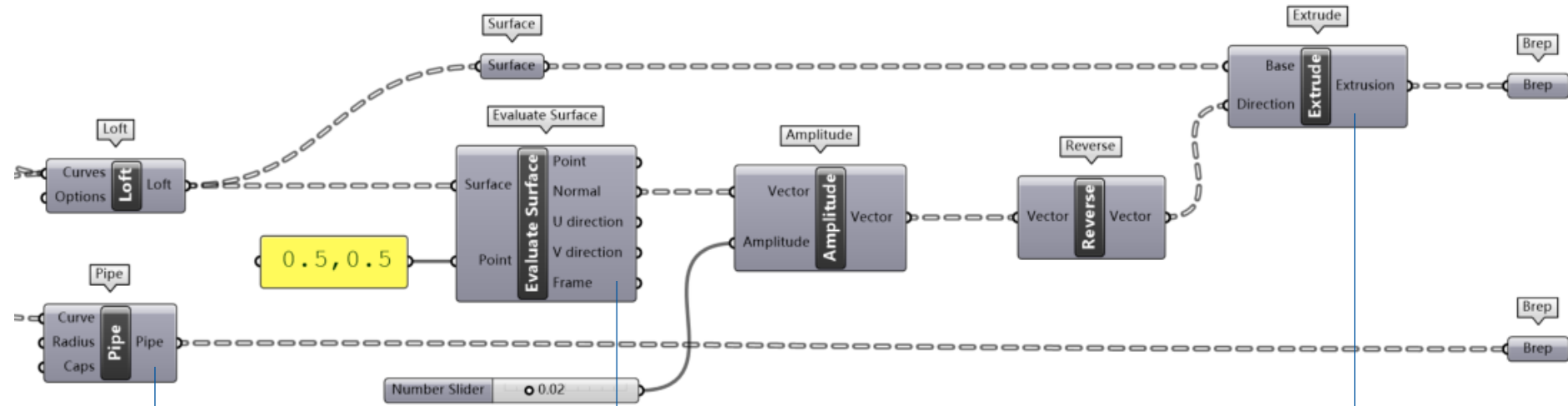
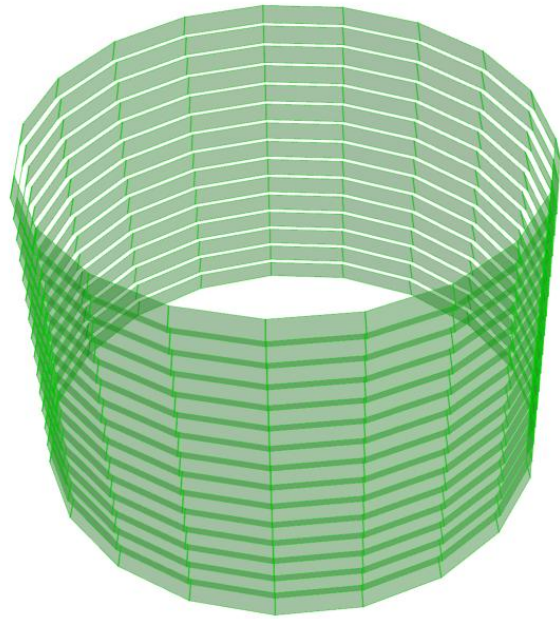
Considering the staggered relationship between the upper and lower parts, an even number is taken.

Take the interval points



Façade elements

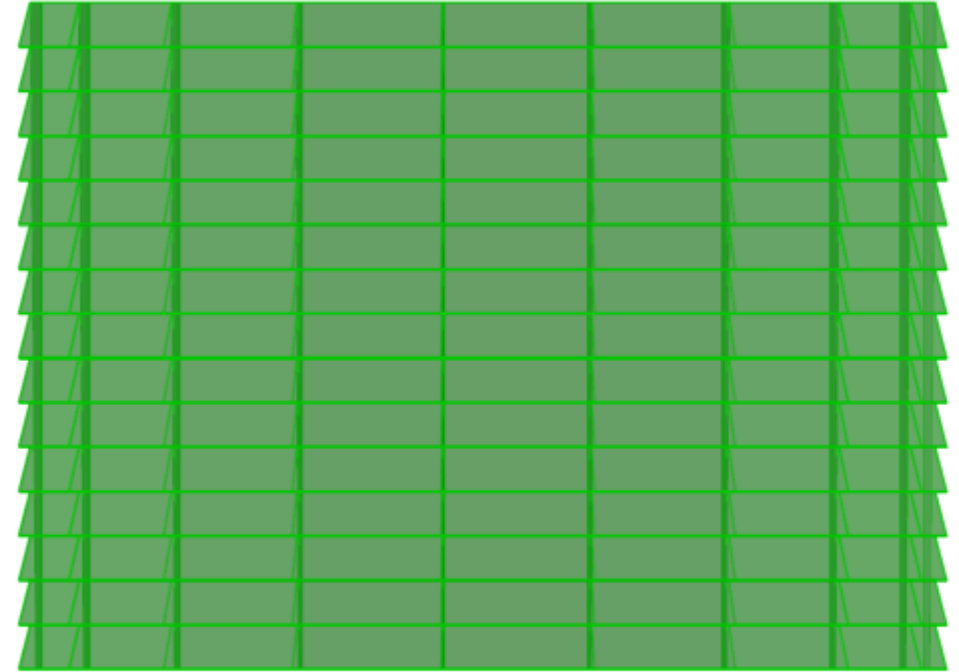
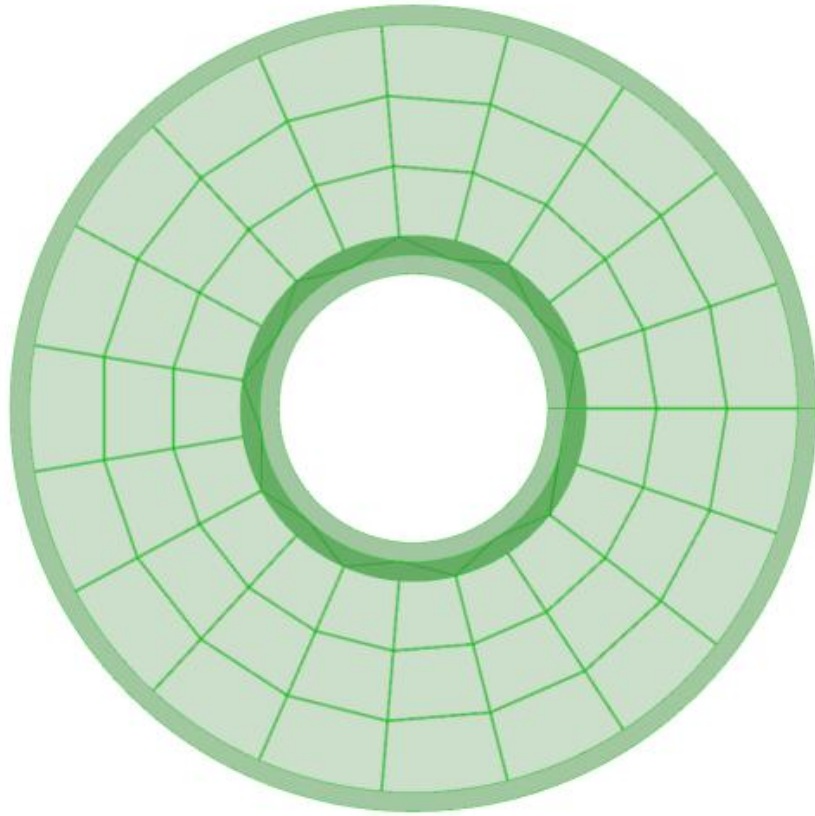
Glass splits



Take the height of both sides of the exploded slope as the splits of the glass

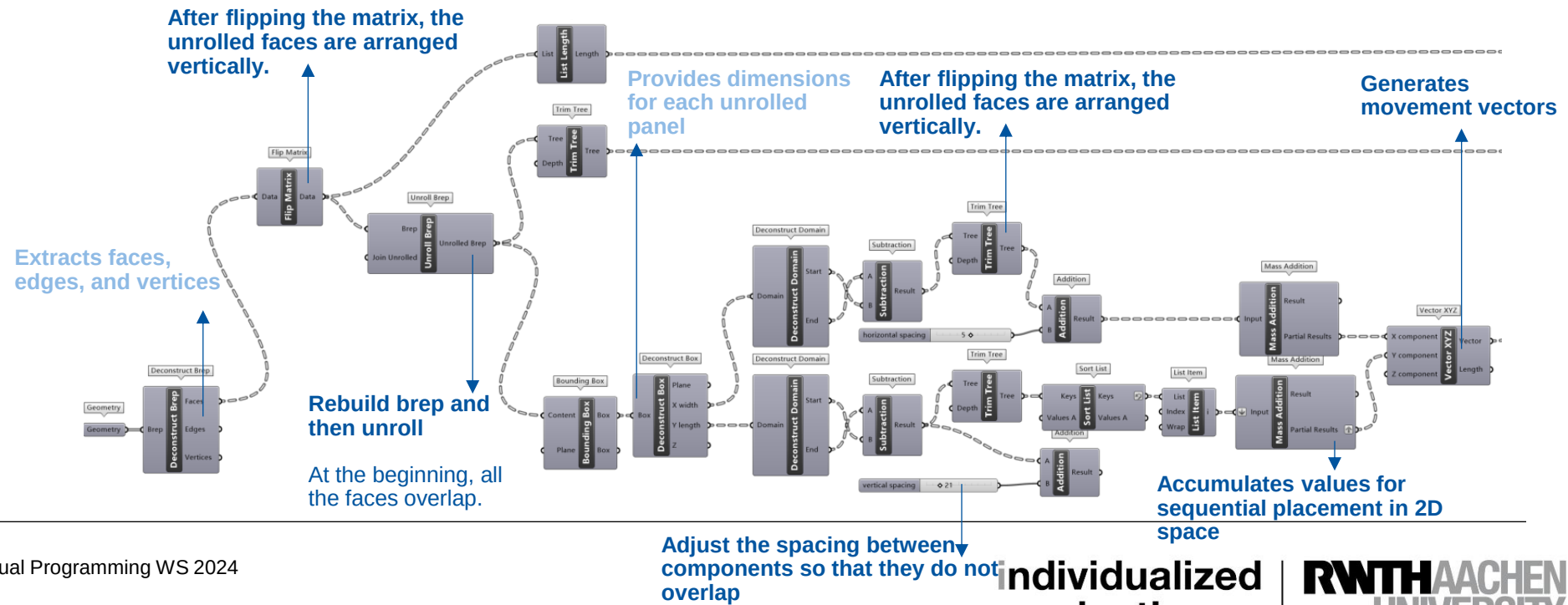
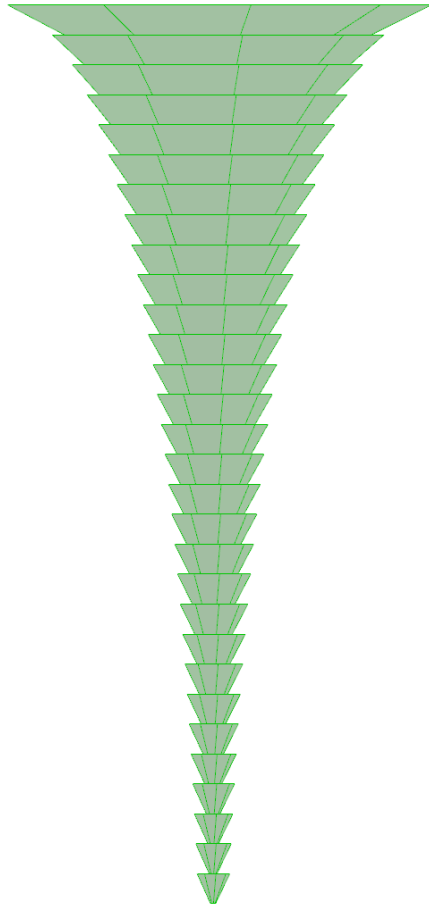
Extract all normal vectors of the slope

Inclined directional internal extrusion to generate thickness



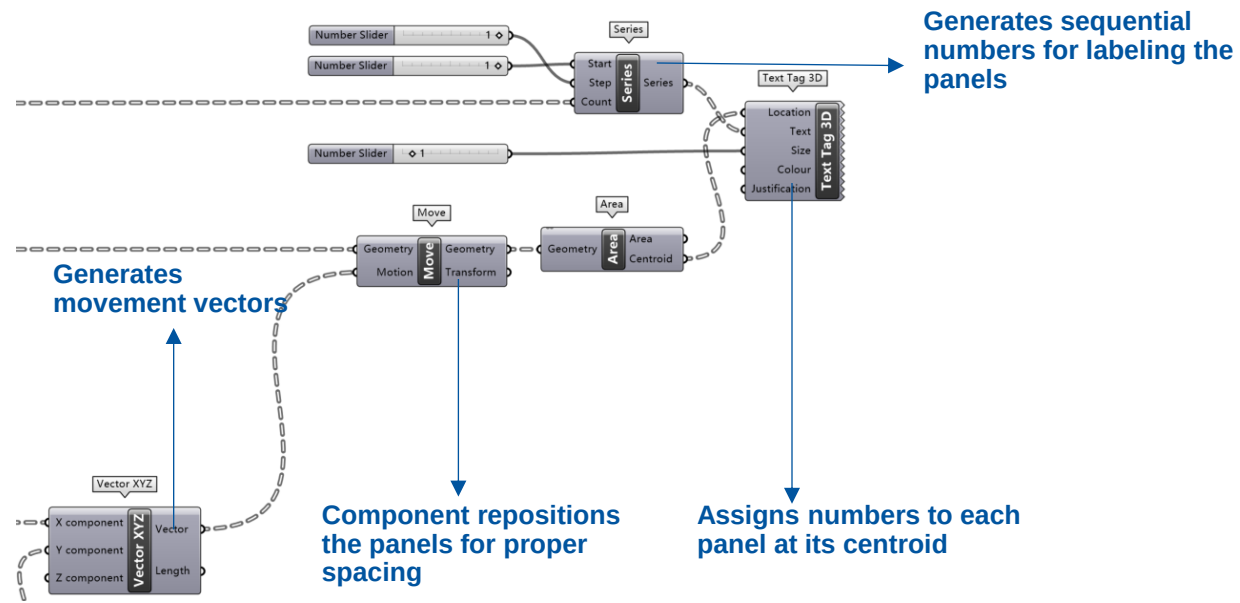
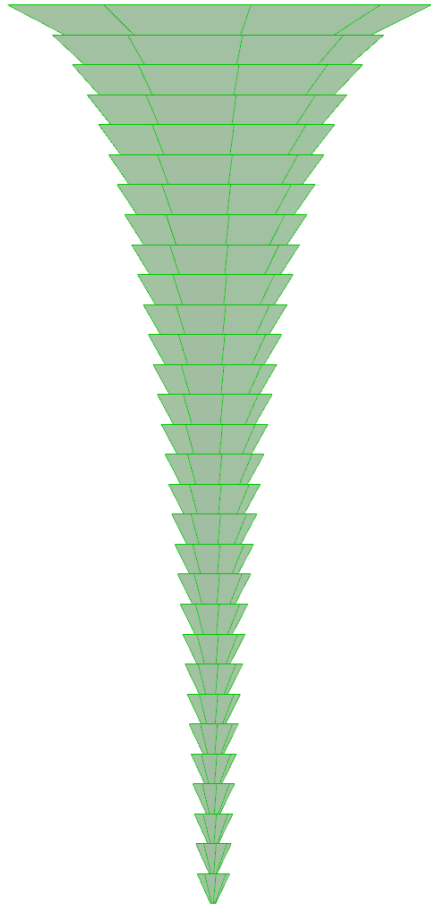
Unfolding 2D data

For the inner mirror cone



Unfolding 2D data

For the inner mirror cone

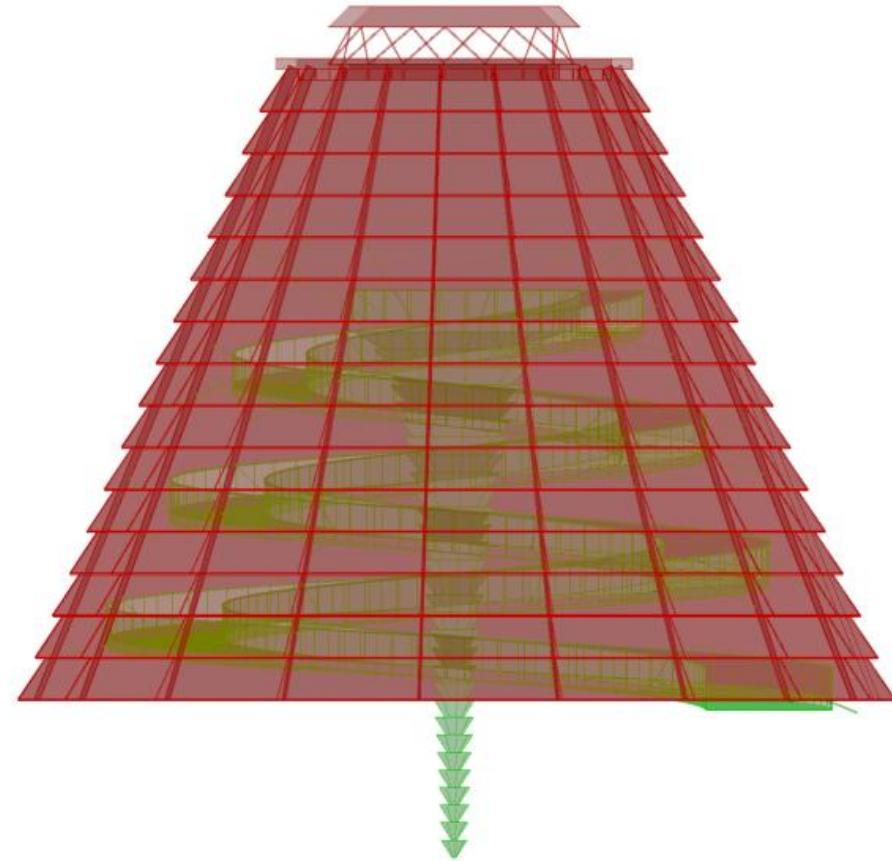


2. Cone

The difference in design between a cone and a cylinder lies in the fact that a cone does not have the complete roof structure of a cylinder. The other parts are basic same.

Additionally, the data structure corresponding to the direction of the braces and inclined girders is also different. I do not have the capability to simplify the data structure to express it through a repetitive basic cell unit, so I am unable to move correctly in response to data changes.

It's too big to show in the presentation, please check the cone.gh file



Python Script

Parametric Shape Generation - Cylinder

```

1 import rhinoscriptsyntax as rs
2 import Rhino.Geometry as rg
3 import math
4 import ghpythonlib.treehelpers as gt
5
6 # Define the interval for height (vertical range) and circular angle (horizontal range)
7 VD = rs.CreateInterval(0, 10) # Height range
8 HD = rs.CreateInterval(0, 2 * math.pi) # Full circle range
9
10 # Function to generate evenly spaced values within a given range
11 def myRange(domain, steps):
12     start = domain[0]
13     end = domain[1]
14     step_size = (end - start) / steps
15     values = [start + i * step_size for i in range(steps + 1)]
16     return values
17
18 # Function to convert a list into a Data Tree structure
19 def Graft(List):
20     myTree = []
21     for i in range(len(List)):
22         myTree.append([List[i]]) # Each item is placed in its own branch
23     return gt.list_to_tree(myTree)
24
25 # Generate vertical and horizontal ranges
26 V_Steps = int(VD.Steps) # Vertical divisions
27 H_Steps = int(HD.Steps) # Horizontal divisions
28
29 VD_Range = myRange(VD, V_Steps)
30 HD_Range = myRange(HD, H_Steps)
31
32 # Convert ranges to Data Tree structures
33 VD_Tree = Graft(VD_Range)
34 HD_Tree = Graft(HD_Range)

```

```

36 # Function to calculate X coordinates for each point on the circular layers
37 def X_Coordinates(base_radius, top_radius, u, v):
38     myTreeList = []
39     for i in range(u.BranchCount):
40         myBranchList = u.Branch(i)
41         current_radius = base_radius + (top_radius - base_radius) * (myBranchList[0] / VD[1])
42         myTemplist = []
43         for k in range(len(v)):
44             x = current_radius * math.cos(v[k])
45             myTemplist.append(x)
46         myTreeList.append(myTemplist)
47     return gt.list_to_tree(myTreeList)
48
49 # Function to calculate Y coordinates for each point on the circular layers
50 def Y_Coordinates(base_radius, top_radius, u, v):
51     myTreeList = []
52     for i in range(u.BranchCount):
53         myBranchList = u.Branch(i)
54         current_radius = base_radius + (top_radius - base_radius) * (myBranchList[0] / VD[1])
55         myTemplist = []
56         for k in range(len(v)):
57             y = current_radius * math.sin(v[k])
58             myTemplist.append(y)
59         myTreeList.append(myTemplist)
60     return gt.list_to_tree(myTreeList)
61
62 # Function to calculate Z coordinates for each point along the height
63 def Z_Coordinates(height, u, v):
64     myTreeList = []
65     for i in range(u.BranchCount):
66         myBranchList = u.Branch(i)
67         myTemplist = []
68         for k in range(len(v)):
69             z = myBranchList[0] * height / u.Branch(u.BranchCount - 1)[0] # Normalize Z coord
70             myTemplist.append(z)
71         myTreeList.append(myTemplist)
72     return gt.list_to_tree(myTreeList)
73

```

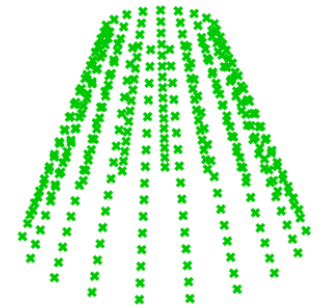
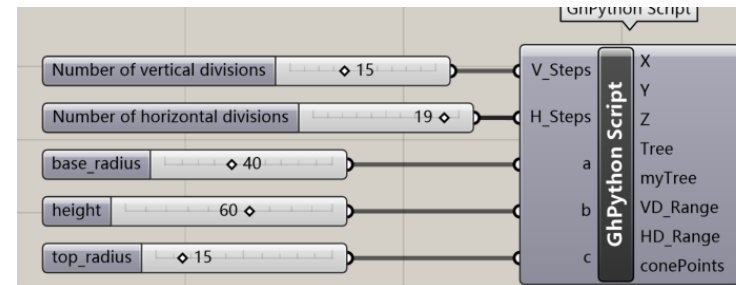
```

74 # Input values for the base radius, top radius, and height of the cone
75 base_radius = float(a)
76 top_radius = float(c)
77 height = float(b)
78
79 # Generate X, Y, Z coordinate trees
80 X = X_Coordinates(base_radius, top_radius, VD_Tree, HD_Range)
81 Y = Y_Coordinates(base_radius, top_radius, VD_Tree, HD_Range)
82 Z = Z_Coordinates(height, VD_Tree, HD_Range)
83
84 # Function to generate 3D points from the coordinate trees
85 def genPoints(x_tree, y_tree, z_tree):
86     myPointTree = []
87     count_1 = x_tree.BranchCount # Number of branches
88     count_2 = len(x_tree.Branch(0)) # Number of points in each branch
89
90     for i in range(count_1):
91         myTemplist_X = x_tree.Branch(i)
92         myTemplist_Y = y_tree.Branch(i)
93         myTemplist_Z = z_tree.Branch(i)
94         myPointList = []
95
96         for k in range(count_2):
97             myTempPoint = rg.Point3d(myTemplist_X[k], myTemplist_Y[k], myTemplist_Z[k])
98             myPointList.append(myTempPoint)
99
100     myPointTree.append(myPointList)
101
102     return gt.list_to_tree(myPointTree)
103
104 # Generate the final tree of points for the conical shape
105 conePoints = genPoints(X, Y, Z)
106
107

```



1. Generate a parametric point cloud by discretizing height and circular divisions.
2. Adjust the base and top radius dynamically to calculate X, Y, and Z coordinates using mathematical functions.
3. Output a structured data tree of points for creating a mesh or NURBS surface in Grasshopper.



5 variants

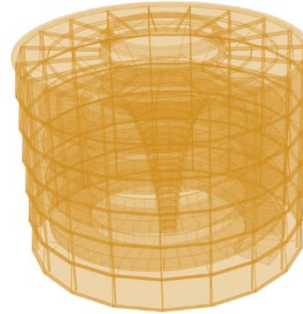
1. Cylinder



Number of Turns



Number of horizontal divisions
9



Number of vertical divisions



Top radius



Base radius

2. Cone

